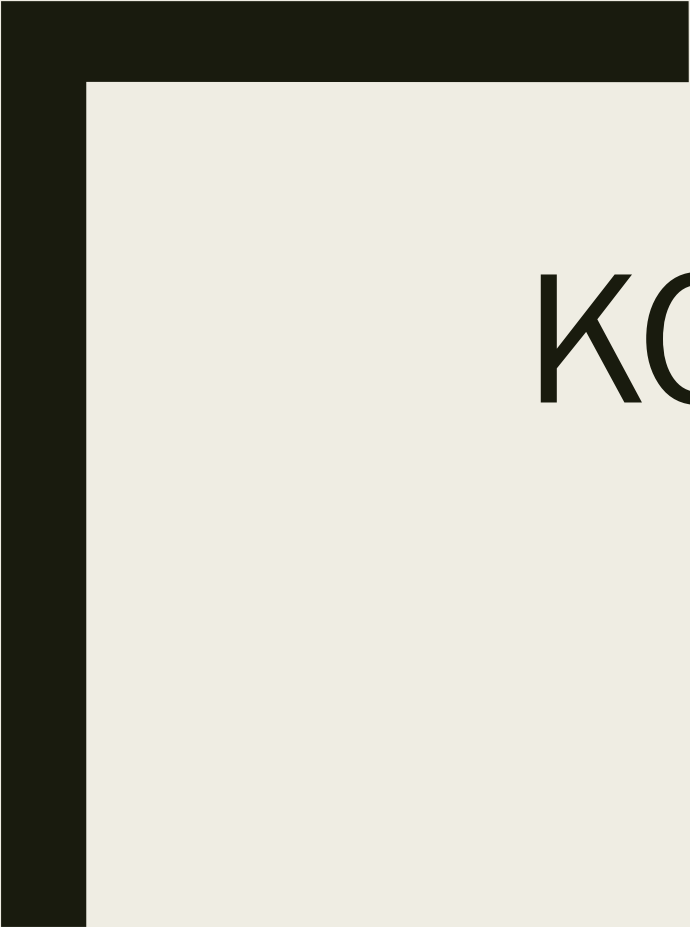




KOMUNIKUJÍCÍ SYSTÉMY



Systémy s předáváním zpráv

- Dvě základní primitiva
- Párová komunikace
 - *send(channel, msg)*
 - *receive(channel, msg)*
- Kolektivní komunikace
 - *broadcast* (vícenásobný send po více komunikačních kanálech)
 - *reduce* (vícenásobný příjem s provedením operace)
- Komunikační kanál, buffering
 - *Asynchronní kanál*
 - neomezená kapacita
 - operace `send` je neblokující
 - složitější implementace
 - příklad: e-mail
 - *Synchronní kanál*
 - omezená kapacita X nulová kapacita
 - `send` může být blokující
 - počet příjemců
 - *Jeden příjemce - kanál*
 - *více příjemců - mailbox*

VŠESMĚROVÉ VYSÍLÁNÍ



Všesměrové vysílání, n-tity

m : zpráva z množiny zpráv msgs

operace **b'cast(m)**, **deliver(m)**

Každá zpráva obsahuje následující položky:

- **sender(m)**, identita odesilatele
- **seq(m)**, sekvenční číslo
- **sender(m) = p** and **seq(m) = i** značí, že zpráva **m** je **i** -tou zprávou zaslanou procesem **p**

Nekorektní procesy

- Nekorektní procesy jsou takové, které mohou po spuštění selhat
- Abstrakce nekorektních procesů:
 - ***Crash & stop*** – po selhání procesu neprovádí žádnou činnost (zastaví se)
 - ***Crash & recovery*** – po nějaké době může dojít k obnovení jejich správné činnosti
 - ***Byzantine*** – byzantské procesy, pro které není chování po selhání definováno

Vlastnosti paralelních programů

- V paralelních a tedy i distribuovaných systémech požadované vlastnosti mohou patřit mezi **životné** (liveness), nebo **bezpečné** (safety)
- Požadujeme odpověď na požadavek? Chceme, aby dříve nebo později nastala událost odpovědi → **životná podmínka**
- Chceme, aby byly zprávy doručovány v nějakém pořadí? Nemůže nastat chyba v doručení → **bezpečná podmínka**

Spolehlivé všesměrové vysílání

- **Platnost (Validity)** – pokud zpráva byla nějakým korektním procesem vysílána, pak dříve nebo později ji každý korektní proces doručí
- **Shoda (Agreement)** – pokud zpráva byla doručena korektním procesem, pak dříve nebo později bude doručena každým korektním procesem
- **Integrita (No duplication nebo Integrity)** – žádná zpráva není doručena více než jednou
- **Opravdovost (No creation)** – pokud process doručil zprávu m od procesu p , potom tento proces zprávu opravdu odeslal
- Budme pracovat s **korektně** se chovajícími procesy, i s **nekorektními**, které mohou být ukončeny během komunikace (crash-stop procesy)

“Best effort” všesměrové vysílání

b'cast(m):

Tag m with sender(m) and seq(m)

send(m) to all neighbors including self

deliver(m):

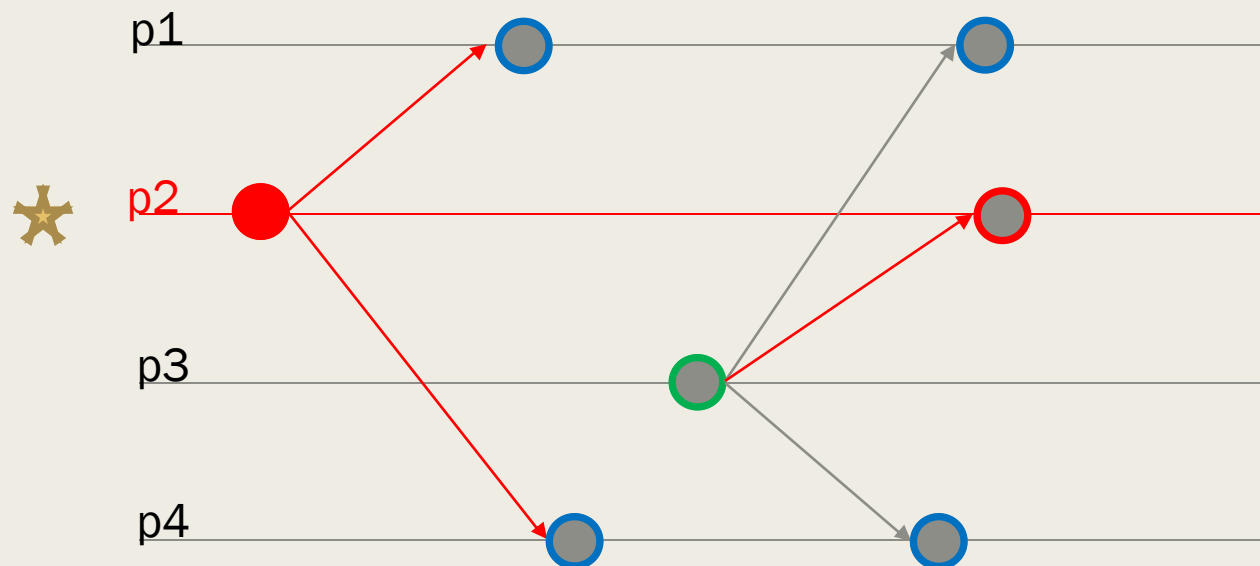
upon **receive**(m) do

 deliver(R , m)

enddo

„Best Effort” všesměrové vysílání

- Je zde garantována **platnost** ale ne **shoda**
- Kdy může nastat situace, že platí **platnost**, ale neplatí **shoda**? V systému nemusí existovat pouze korektní procesy. Pak **vysílá** jen jeden korektní proces p_3 a od toho **doručí** zprávu všechny korektní procesy p_1 a p_4
- Proces p_2 není korektní, proto **platnost** nenaruší to, že zpráva není doručena
- Zpráva od p_2 byla **doručena** korektními p_1 a p_4 , pak pro **shodu** musí být **doručena** i p_3 , což neplatí! Nedoručení zprávy od p_3 procesu p_2 ovšem **shodu** nenaruší



Předpoklady pro komunikaci při všesměrovém vysílání

■ Komunikace

- *Známé největší možné zpoždění při doručování zprávy*
- *Lokální hodiny pro každý z procesů*
- *Známý nejvyšší časový limit pro vykonání interní akce procesem v jednom kroku.*

■ Topologie

- *Topologie sítě je obecná*

Spolehlivé (reliable) všesměrové vysílání

b'cast(R, m):

Tag m with sender(m) and seq(m)

send(m) to all neighbors including self

deliver(R, m):

upon **receive**(m) **do**

if p has not previously executed deliver(R, m) **then**

if sender(m) $\neq p$ **then** **send**(m) to all neighbours

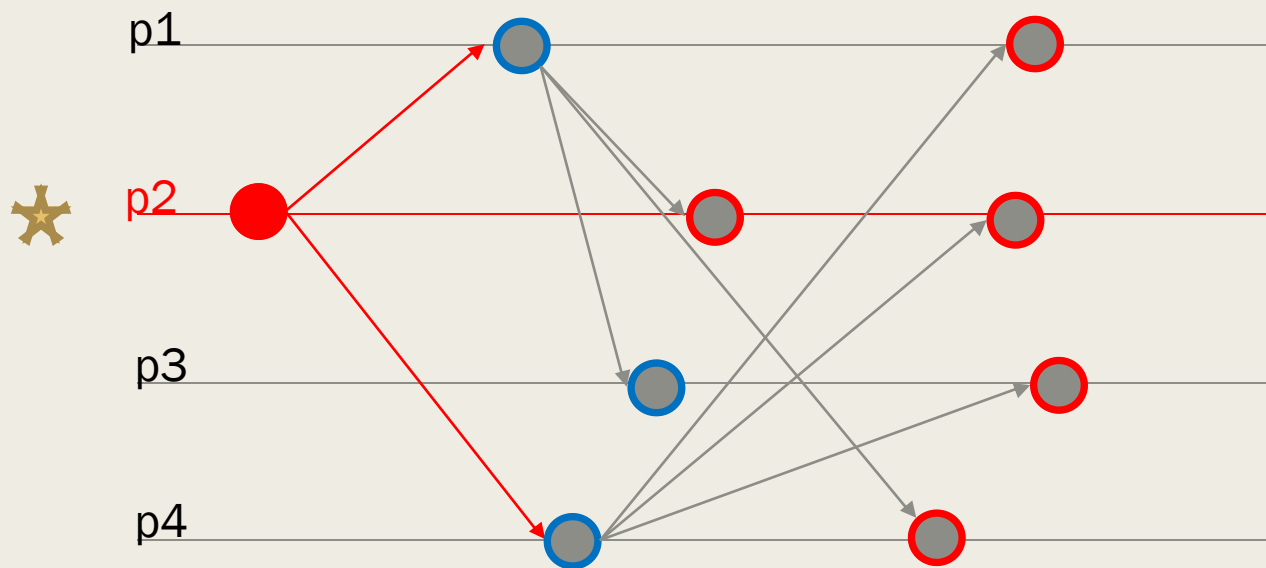
deliver(R, m)

endif

enddo

Spolehlivé všesměrové vysílání

- Procesy přeposílají obdržené zprávy
- Je zde garantována **platnost** i **shoda**



Další vlastnosti všesměrového vysílání

- **1) FIFO uspořádání:** Pokud nějaký proces vysílá zprávu m před tím, než vysílá zprávu n , pak žádný korektní proces nedoručí zpráv n před tím, než doručí zprávu m – *“FIFO order” from single process*
- **2) Kauzální (Causal) uspořádání:** Pokud vysílání zprávy m kauzálně předchází vysílání zprávy n , pak žádný korektní proces nedoručí zprávu n dokud nedoručí zprávu m
- **3) Úplné (Total) uspořádání:** Pokud korektní procesy p a q oba doručí zprávy m a n , pak p doručí m before n iff q delivers m before n

Přijímají z pohledu příjemců ve stejném pořadí všechny procesy

Třídy všesměrového vysílání

- $\text{Reliable} = \text{Validity} + \text{Agreement} + \text{Integrity}$
- $\text{FIFO} = \text{Reliable} + \text{FIFO Order}$
- $\text{Causal} = \text{Reliable} + \text{Causal Order}$
- $\text{Atomic} = \text{Reliable} + \text{Total Order}$
- $\text{FIFO Atomic} = \text{Reliable} + \text{FIFO Order} + \text{Total Order}$
- $\text{Causal Atomic} = \text{Reliable} + \text{Causal Order} + \text{Total Order}$

FIFO všesměrové vysílání

Initialize:

forall q

msgbag[q] := empty

next[q] := 1

b'cast(F, m): b'cast(R, m)

deliver(F, m):

upon deliver(R, m) do

q := sender(m)

msgbag[q] := msgbag[q] $\cup$ { m }

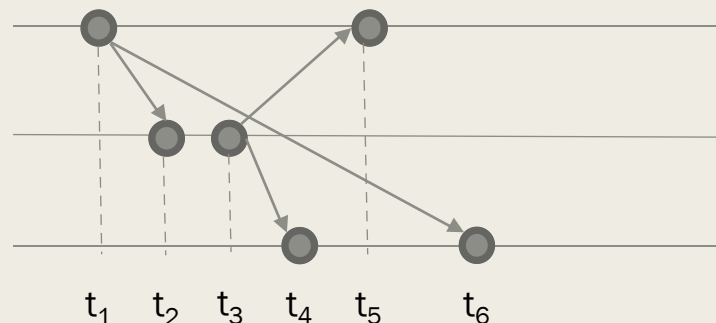
while exists message n in msgbag[q] with seq(n) = next[q] do

deliver(F, n); next[q] := next[q] + 1

msgbag[q] := msgbag[q] - { n }

enddo

Kauzální předávání zpráv



t_1 : David posílá zprávu Petrovi a Karlovi: „Já a Karel chceme vědět, jak hrála Zbrojovka na Slávii“

t_2 : Petr doručuje zprávu od Davida: „ Já a Karel chceme vědět, jak hrála Zbrojovka na Slávii“

t_3 : Petr posílá zprávu Karlovi a Davidovi: „2:0“

t_4 : Karel doručuje zprávu od Petra: “2:0” *... a Karel je zmaten ... o co jde?*

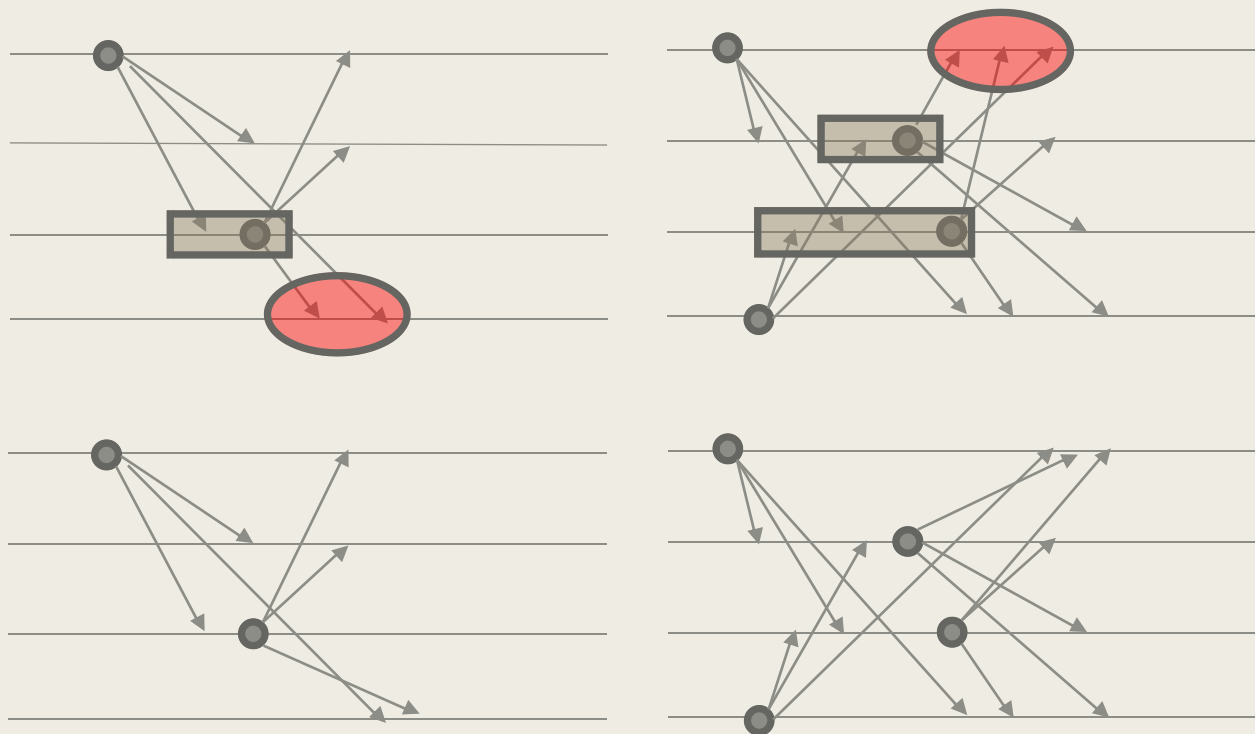
t_5 : David doručuje zprávu od Petra: “2:0”

t_6 : Karel doručuje zprávu od Davida: „ Já a Karel chceme vědět, jak hrála Zbrojovka na Slávii“

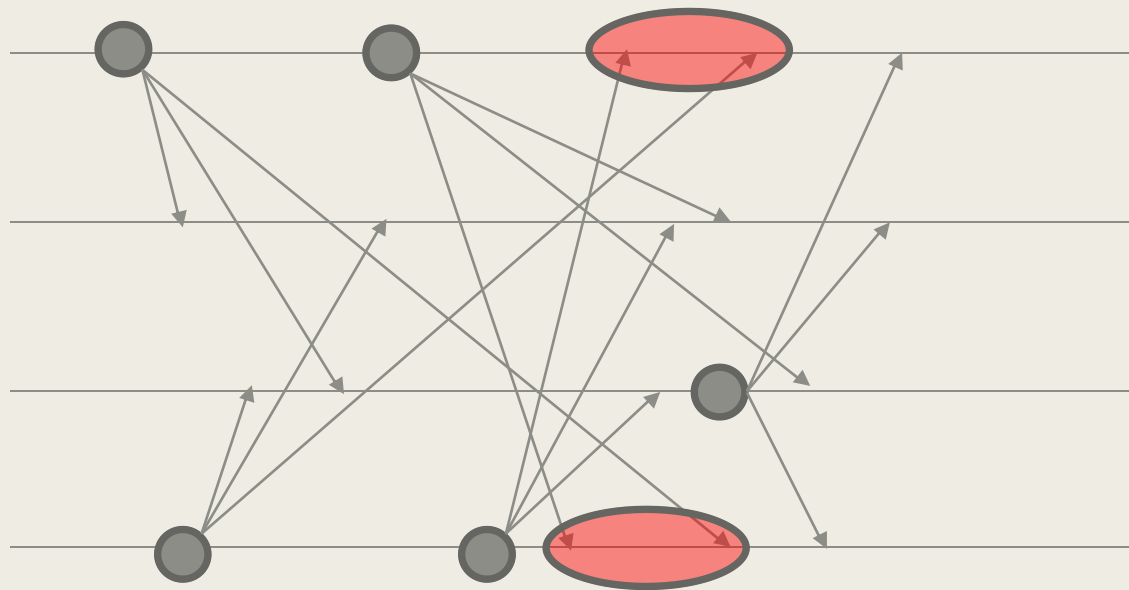
Kauzální všesměrové vysílání

- Zavedeme relace *kauzality*
- Relace $\rightarrow_e$ kauzálně předchází, když
 - $a \rightarrow_e b$ pokud jeden process vykonal tyto události v tomto pořadí
 - $broadcast(ma) \rightarrow_e deliver(ma)$
 - *Tranzitivita*
- Pokud jeden proces doručí zprávu ma a pak vyšle zprávu mb , pak všechny procesy musí doručit ma před mb , nezáleží ale v jakém pořadí doručování zpráv před doručením mb probíhá!

Kauzální?



Kauzální, ale ne FIFA



Kauzální všesměrové vysílání

Initialize: prevDlvr := empty

b'cast(C, m):

 b'cast(F, <prevDlvr || m>)

 prevDlvr := empty

deliver(C, m):

 upon deliver(F, <m1, ... mk>) do

 for i := 1 to k do

 if p has not previously executed deliver(C, mi)

 then

 deliver(C, mi)

 prevDlvr := prevDlvr || mi

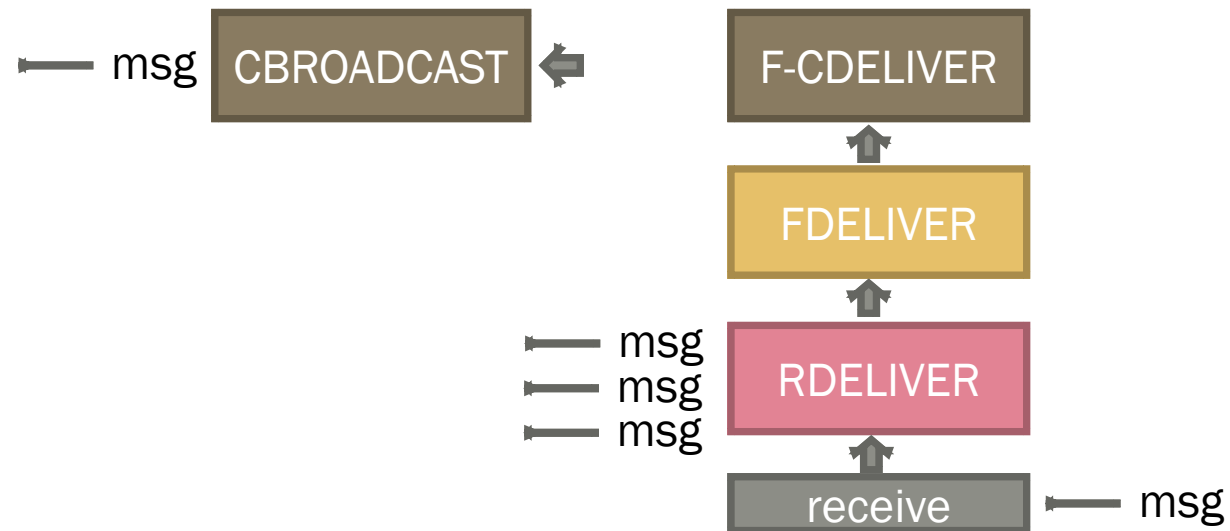
 endif

 enddo

Kauzální vysílání a doručení

`broadcast(C,m)` // volá se pro C-bcast

`deliver(C,m)` // reaguje na přijetí (FIFA-CAUSAL doručení)



Atomické všesměrové vysílání (ABCAST)

- ABCAST odesílatel p zasílá $m(p,t)$ všem ostatním procesům
- Každý příjemce q přidá zprávu do $bag[p]$, označí ji “**U**” a přiřadí ji hodnotu vyšší než doposud zjištěné priority (tedy buď přiřazené tímto procesem dříve, nebo obdržené s rozhodnutím **D**, viz níže)
- Každý příjemce informuje odesílatele o přidělené hodnotě zprávě m , “**U**” – hodnota odeslaná odesílateli – “**D**” – hodnota upravená odesílatelem
- Odesílatel sesbírá všechny odpovědi na svoji zprávu, zjistí maximální přiřazenou hodnotu a o té informuje ostatní procesy
- Tyto si hodnotu k procesu poznačí jako „**D**“
- Doručí všechny zprávy podle hodnot dokud je nějaká zpráva v bagu, nebo nějaká zpráva s aktuálně nejnižší hodnotou je označena jako “**U**”

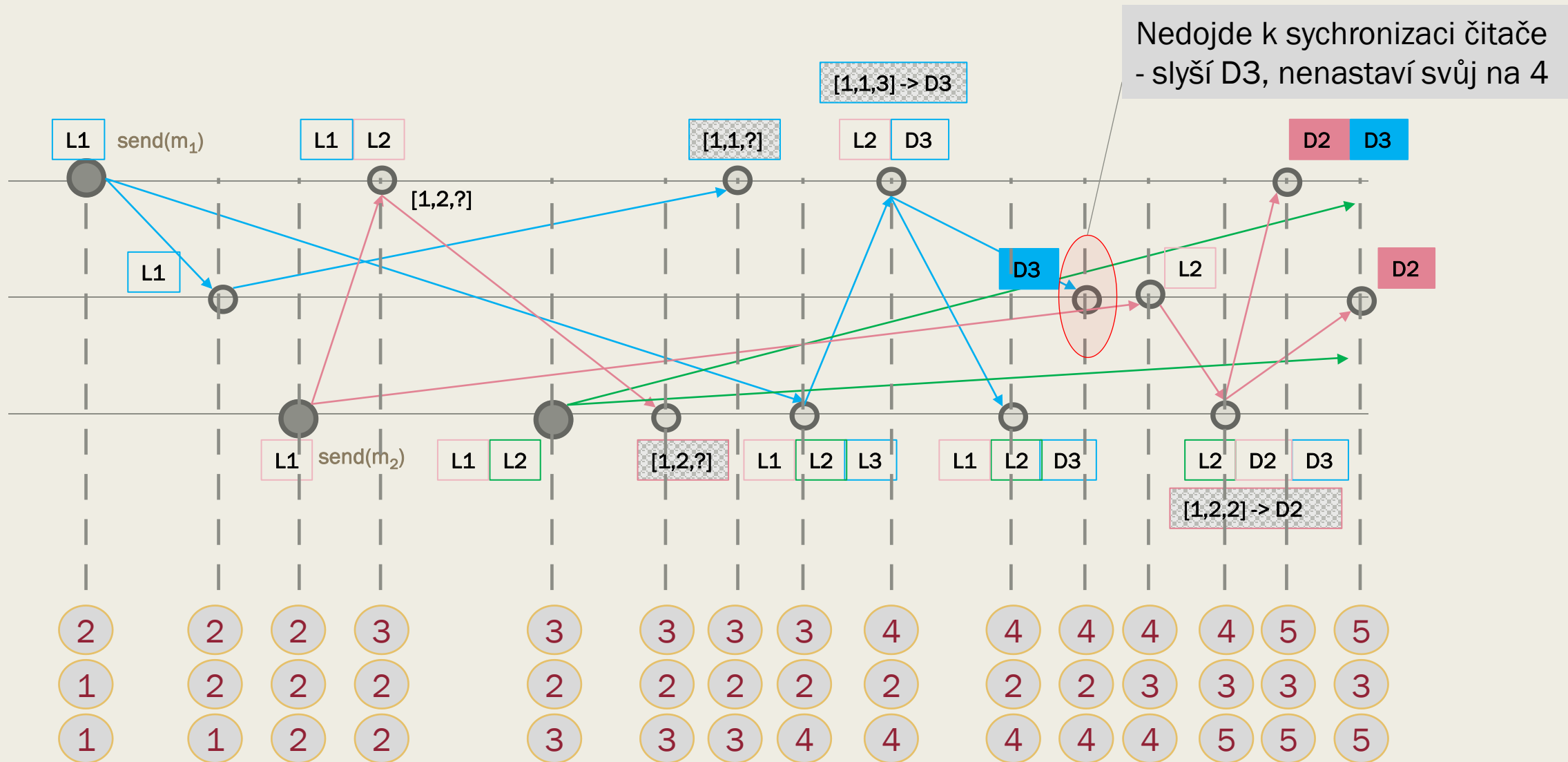
ABCAST – stejná hodnota, rozhodnutí ?

- P1[1]: **bcast**($m_1, (p_1, 1)$) ; P2[1]: **bcast**($m_2, (P_2, 1)$)
- P1[2]: **send**($P_2, U(m_1, 2)$); P2[2]: **send**($P_1, U(m_2, 2)$);
- P1[3]: **send**($P_2, D(m_1, 2)$); P2[3]: **send**($P_1, D(m_2, 2)$)
- M1 a m2 mají stejnou hodnotu konsensu =>
- V tom případě se doručí podle indexu procesu a časového razítka, které přidělil vysílající proces (tj. sekundární klíče jsou nejprve číslo procesu a pak časové razítko, které dal vysílající proces zprávě).

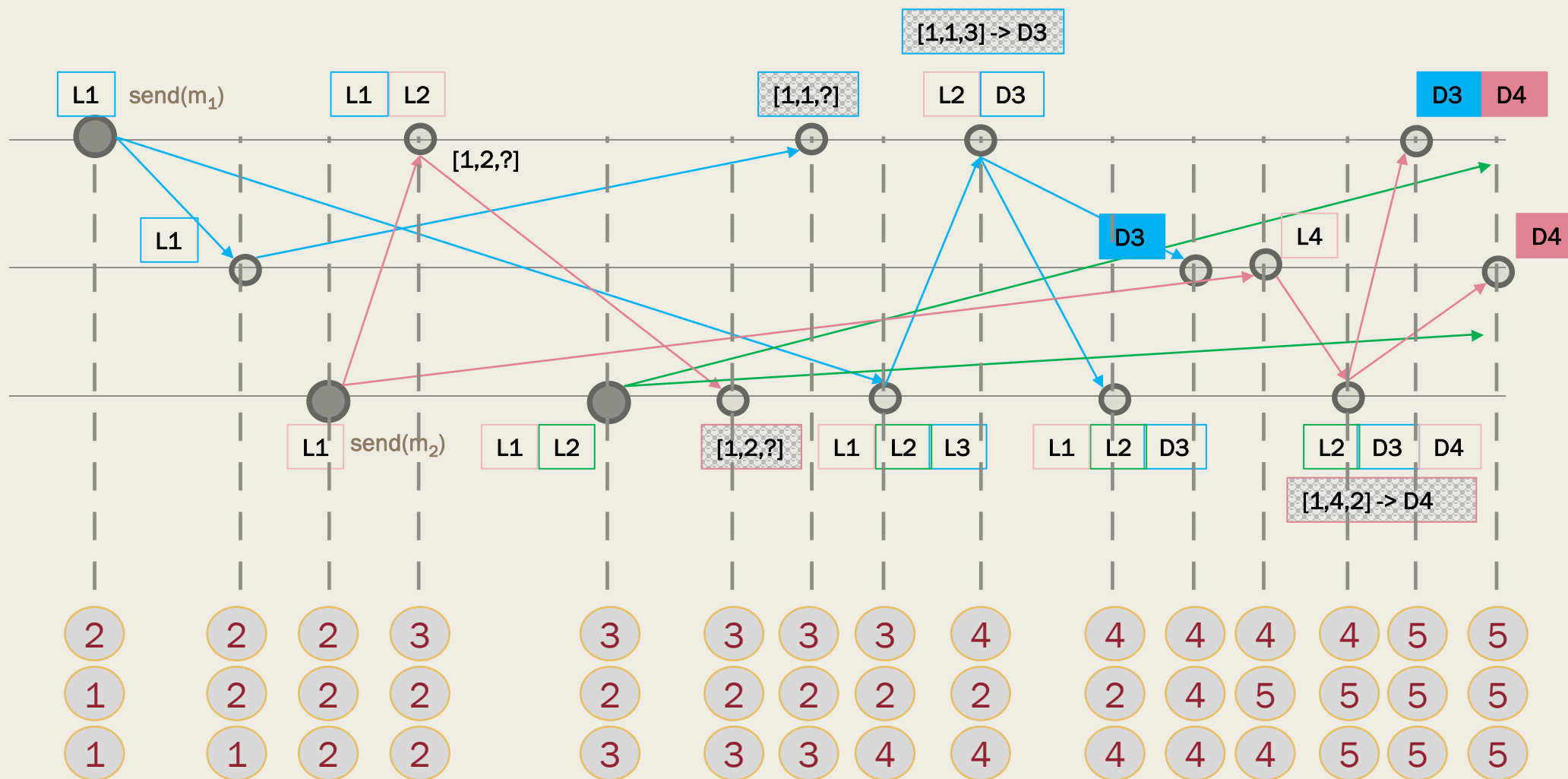
ABCAST – náznak důkazu

- Zpráva m s nejnižším rozhodnutím je nějakým procesem P doručena, a přitom přijde rozhodnutí o zprávě m' , že má vyšší nebo stejnou hodnotu doručení, než má m (???)
- *A, tato zpráva ještě nebyla procesu P předána spolehlivým broadcastem, potom ale dostane vyšší číslo U od tohoto procesu (výsledná priorita musí být tím pádem nižší).*
- *B, proces již poslal své U pro tuto zprávu a to je nižší nebo stejné než je D pro zprávu m -> potom musí být ale ve frontě doručovaných zpráv záznam pro m' před m , což není*
- *Pozn: fronta zpráv k doručení řadí U před L pro stejné časové razítko, nebo alespoň podle sekundárních klíčů!*

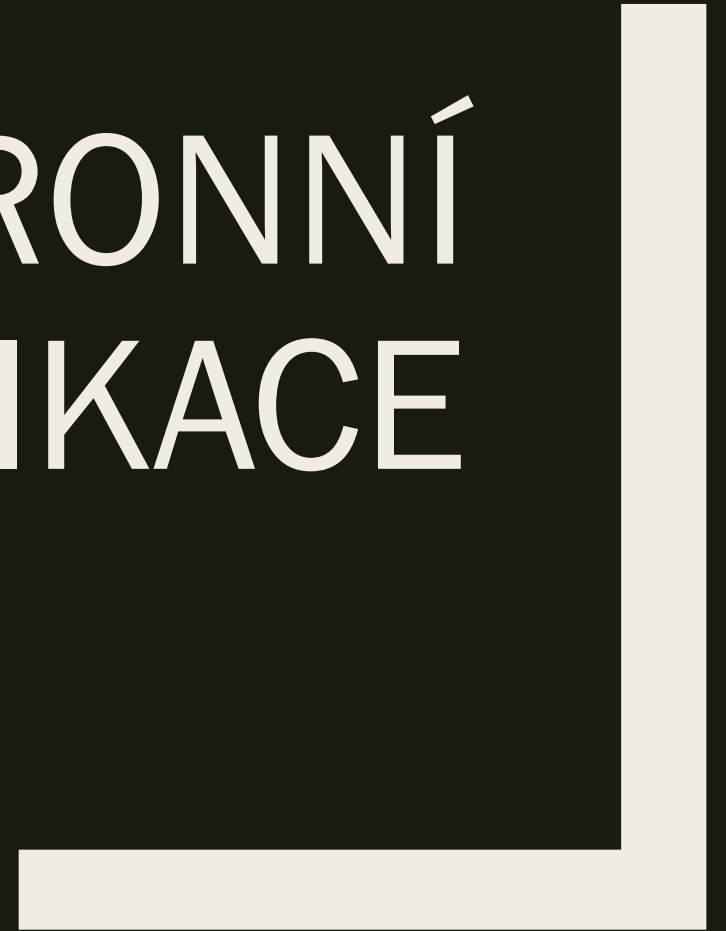
ABCAST, příklad – nefunkční řešení



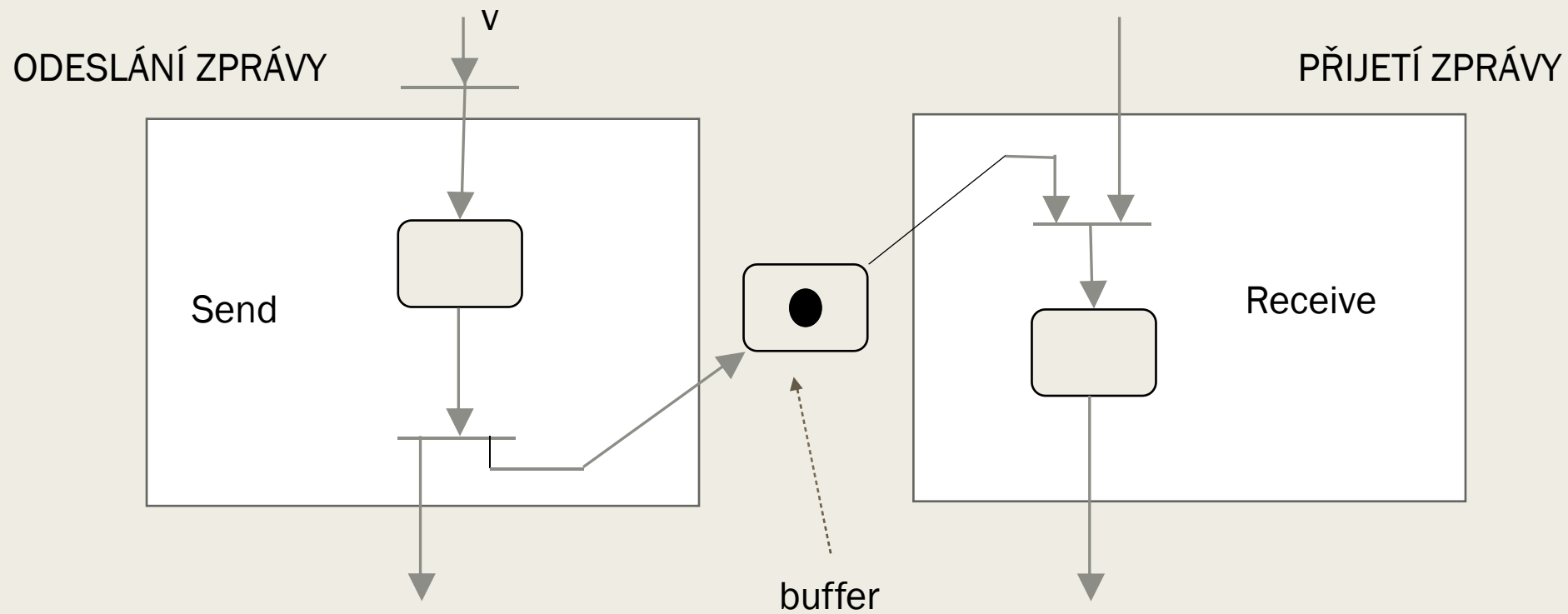
ABCAST, příklad



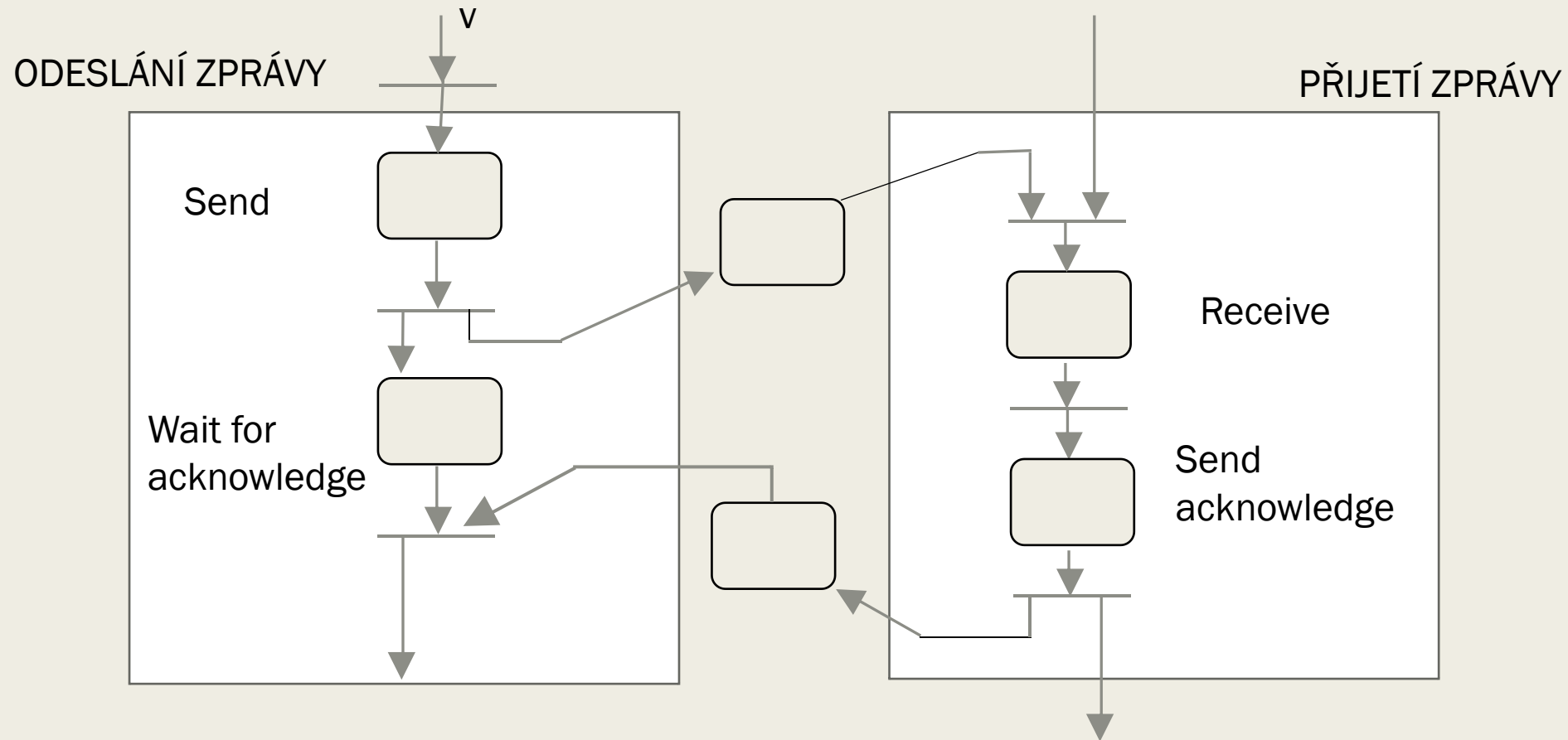
SYNCHRONNÍ KOMUNIKACE



Asynchronní komunikace



Synchronní komunikace



Simulace synchronního kanálu skrz asynchronní

■ Nulová kapacita

- `send(ch, msg) ⇒`
`send(ch1, msg)`
`receive(ch2, dummy)`
- `receive(ch, msg) ⇒`
`receive(ch1, msg)`
`send(ch2, „ack“)`

■ Nenulová kapacita (n zpráv)

- `init(ch) ⇒`
`n_times {`
`send(ch2, „ack“)`
`....`
`send(ch2, „ack“)`
`}`

Synchronní komunikace

- D je funkce zobrazující množinu událostí v systému na diskrétní množinu ‘fyzického času’
- Pak pro synchronní komunikaci platí
 - **Platnost** – pokud process $p1$ synchronně obdrží zprávu od $p2$, pak ji process $p2$ synchronně odeslal
 - **Integrita** – žádný proces synchronně neobdrží zprávu víc než jedenkrát
 - **Synchronnost**
 - Pokud $e1 \rightarrow_e e2$, pak $D(e1) < D(e2)$
 - $D(\text{send}(m)) = D(\text{receive}(m))$
 - **Ukončení**
 - Každá synchronně odeslaná zpráva bude synchronně doručena

Proveditelnost programu v systému se synchronním kanálem

- Synchronní komunikace je kauzální, ale kauzální nemusí být synchronní
- Program pro asynchronní systém (A-execution) je proveditelný v systému se synchronní komunikací (RSC, realizable with synchronous communication) pokud neobsahuje **korunu** (crown)

Koruna velikosti k je, když

send(m_1) $\rightarrow_e$ **receive**(m_2)

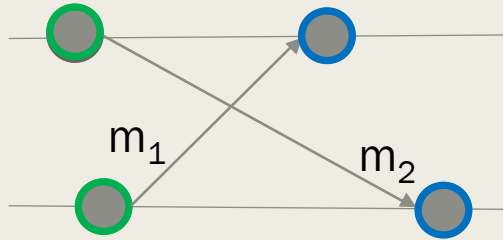
send(m_2) $\rightarrow_e$ **receive**(m_3)

...

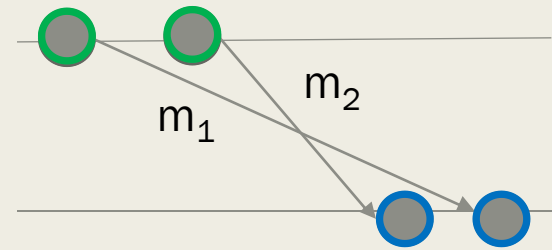
send(m_{k-1}) $\rightarrow_e$ **receive**(m_k)

send(m_k) $\rightarrow_e$ **receive**(m_1)

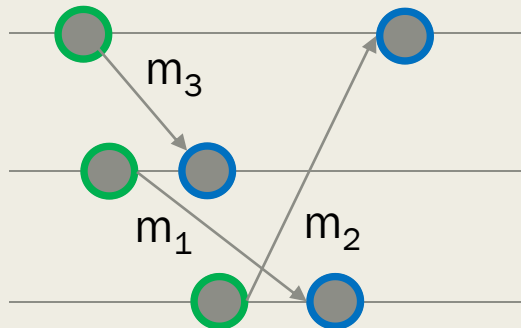
Koruna (příklady velikostí 2 a 3)



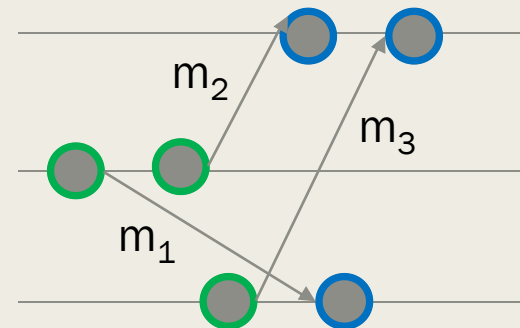
$\text{send}(m_1) \rightarrow_e \text{receive}(m_2)$
 $\text{send}(m_2) \rightarrow_e \text{receive}(m_1)$



$\text{send}(m_1) \rightarrow_e \text{receive}(m_2)$
 $\text{send}(m_2) \rightarrow_e \text{receive}(m_1)$



$\text{send}(m_2) \rightarrow_e \text{receive}(m_1)$
 $\text{send}(m_1) \rightarrow_e \text{receive}(m_3)$
 $\text{send}(m_3) \rightarrow_e \text{receive}(m_2)$



$\text{send}(m_3) \rightarrow_e \text{receive}(m_1)$
 $\text{send}(m_1) \rightarrow_e \text{receive}(m_2)$
 $\text{send}(m_2) \rightarrow_e \text{receive}(m_3)$

Proveditelnost asynchronního programu synchronním

Program napsaný pro systém s asynchronní komunikací je proveditelný systémem se synchronní komunikací, pokud v něm neexistuje koruna

Pokud $\text{send}(m_1) \rightarrow_e \text{receive}(m_2)$, pak $(m_1, m_2) \in G$

Jelikož není koruna, pak nemůžou v G existovat cykly:

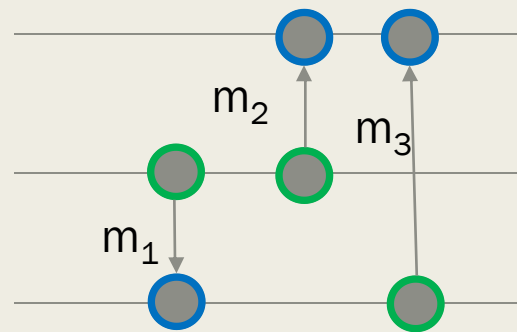
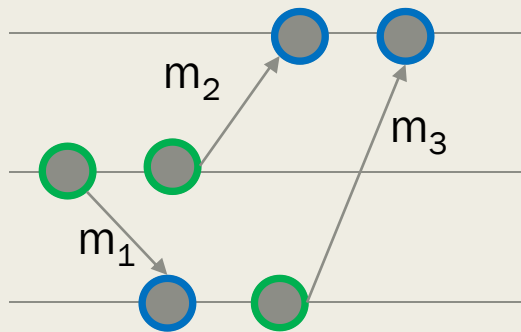
... jakýkoliv cyklus v G $(m_1, m_2, m_3 \dots m_k, m_1)$ znamená, že v G jsou hrany

$(m_1, m_2), (m_2, m_3) \dots (m_k, m_1)$

a tedy i koruna

$\text{send}(m_1) \rightarrow_e \text{receive}(m_2), \text{send}(m_2) \rightarrow_e \text{receive}(m_3) \dots \text{send}(m_k) \rightarrow_e \text{receive}(m_1)$

Synchronní provedení asynchronní komunikace bez koruny, příklad



send(m_1) $\rightarrow_e$ **receive**(m_2)

send(m_1) $\rightarrow_e$ **receive**(m_3)

send(m_2) $\rightarrow_e$ **receive**(m_3)

$G = \{(m_1, m_2), (m_1, m_3), (m_2, m_3)\} \rightarrow$ splněno pro uspořádání (m_1, m_2, m_3)

Synchronní komunikace klient / server

Klient (process i) je odesílatel

Server

```
wait(may_read[i])  
x ← obtain(i)  
may_readj[i] ← false  
signal(i, end_rdv)
```

Klient

```
buffer ← m  
signal(j, may_read[i])  
wait(end_rdvi)  
end_rdv ← false
```

Klient (process i) je příjemce

Server

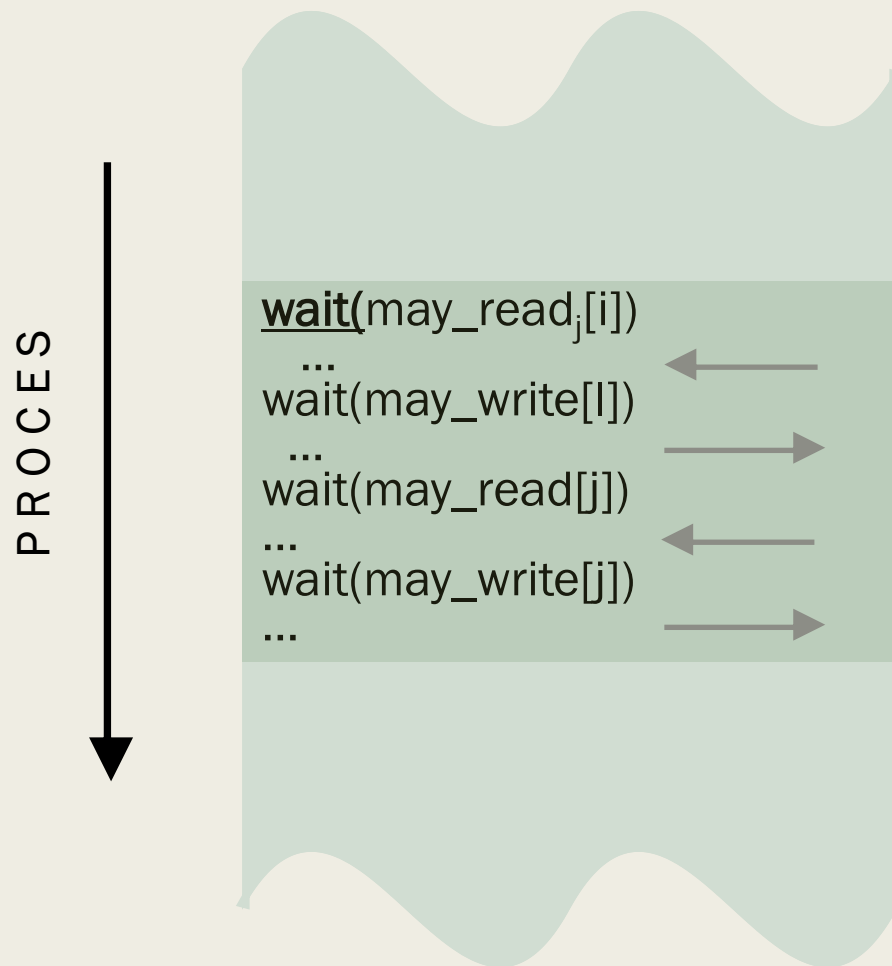
```
wait(may_write[i])  
delivere(i,m)  
may_write[i] ← false  
signal(i, end_rdv)
```

Klient

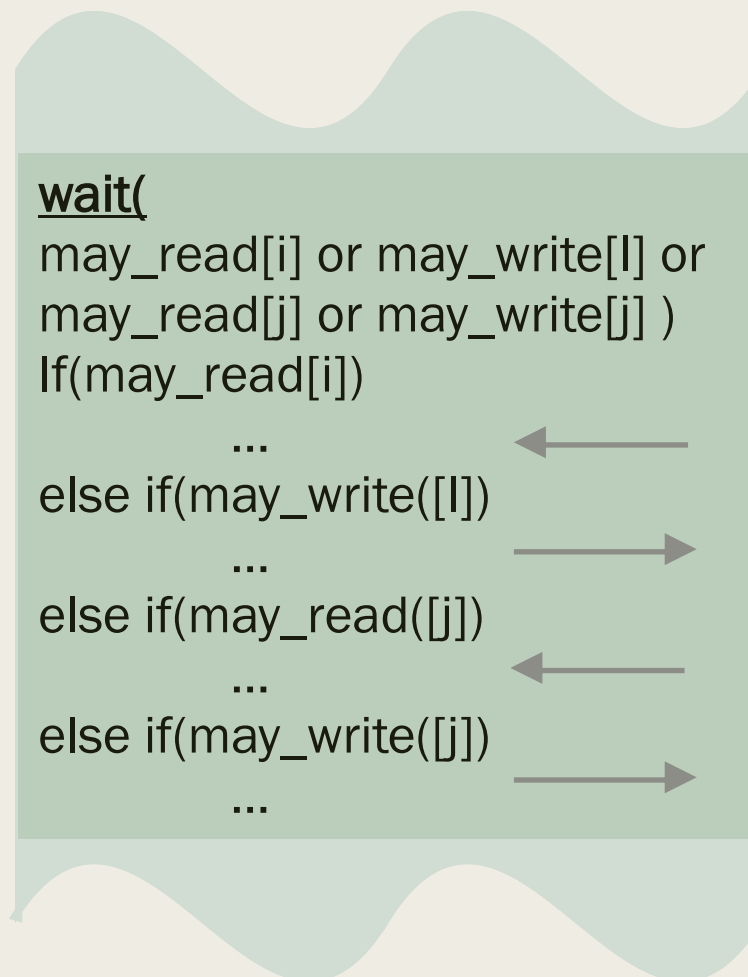
```
signal(j, may_write[i])  
wait(end_rdvi)  
x ← bufferi  
end_rdvi ← false
```

Rendezvous kontext

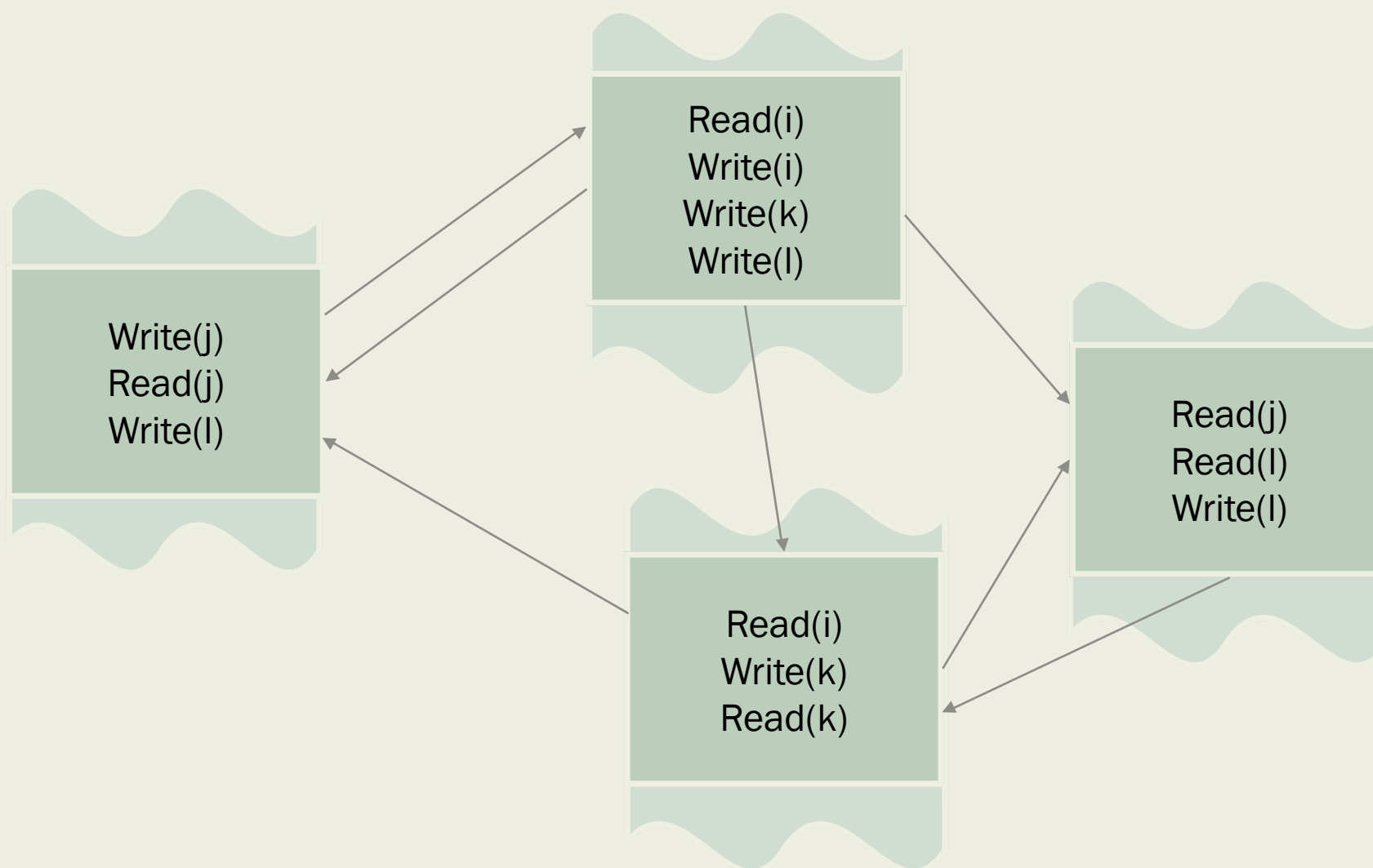
Deterministický rendezvous kontext



Nedeterministický rendezvous kontext



Synchronní komunikace klient / server



Synchronizace rendezvous procesů

- (Bagrodia 1989) - Synchronizace rendezvous procesů, zabránění vzniku koruny,
- Pracuje s tokeny, které reprezentují interakci mezi dvěma procesy
- **Základní princip:**
 - *Aby oba procesy interagovali (= provedli synchronní komunikaci) musí o to mít oba zájem*
 - Jsou v Rendezvous části program
 - Mají zájem interagovat s druhým procesem
 - *Právě jeden ze dvou procesů i a j drží token $TOKEN(\{i, j\}, i)$ (token pro komunikaci mezi procesy i a j je držen procesem i . Tento token slouží pro komunikaci v obou směrech, nemáme tedy $TOKEN(\{j, i\}, i/j)$)*
 - *Procesy jsou prioritně uspořádány, procesy s menším číslem mají větší prioritu*
 - *Proces mimo rendezvous kontext je ve stavu OUT, v tomto kontextu buď ve stavu INTERESTED nebo ENGAGED*
 - *Proces, pokud čeká na návrh interakce, a nějaký jiný proces má zájem o interakci, tak pokud má větší prioritu si tento proces poznačí a odpoví mu až obdrží odpověď, na kterou čeká*

Synchronizace rendezvous procesů

Proces vstupuje do kontextu rendezvous (dále jen kontextu):

1. Proces vstupuje do kontextu, nastavuje svůj stav na “INTERESTED”
2. Pokud drží token pro některý poznačený proces, se kterým chce komunikovat, pošle jej tomuto procesu a nastaví svůj stav na “ENGAGED”
3. Nastaví záznam o alternativním procesu k interakci na prázdný

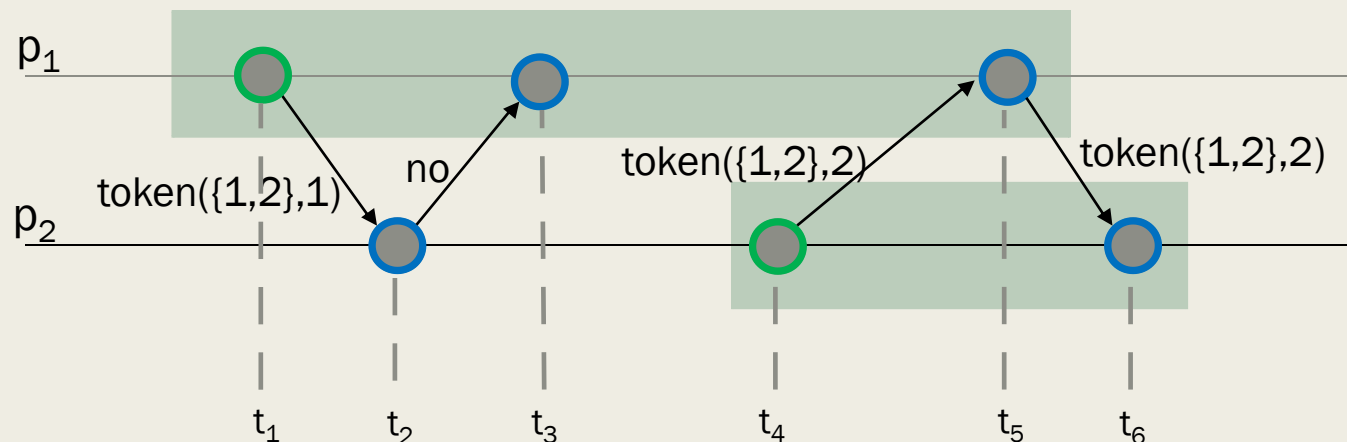
Proces obdržel token:

1. Pokud je mimo kontext ,nebo nemá zájem s tímto procesem intereagovat, pošle mu “no”, token si ponechá
2. Pokud je ve stavu “INTERESTED”, pošle token zpět a dojde k interakci – **rendezvous**. Proces je tímto mimo kontext, Stav je “OUT”
3. Zbývá možnost, že proces je v kontextu a jeho stav je “ENGAGED” (čeká na odpověď na svoji výzvu)
 1. *Pokud je iniciátorem výměny tokenu on sám, pak byla přijata výzva partnerským procesem – **rendezvous**, Proces je tímto mimo kontext, Stav je “OUT”. Navíc pokud má poznačeno, že by se měl ozvat na později mu doručenou nabídku k interakci, pošle zpět zprávu “no” – již interakci má – a token si ponechá*
 2. *Pokud je iniciátorem proces s nižší prioritou – vyšším číslem, a nemá zatím poznačenou alternativní nabídku, poznačí si tuto a zatím nic neodpoví, ani token nevrací*
 3. *Pokud je iniciátorem proces s vyšší prioritou – nižším číslem, nebo již má poznačenou alternativní nabídku k interakci, odpoví “no” a token si ponechá*

Proces obdržel zprávu “no”:

1. Pokud má poznačenou alternativní nabídku, pošle token který s touto nabídkou obdržel zpět – **rendezvous**. Proces je tímto mimo kontext, Stav je “OUT”
2. Pokud nemá, provede opět část “**Proces vstupuje do kontextu**”

Synchronizace rendezvous procesů, příklad



t_1 : proces p_1 je v kontextu, vybírá partnera pro komunikaci p_1 a posílá mu token

t_2 : proces p_2 není v kontextu a proto odpovídá 'no'

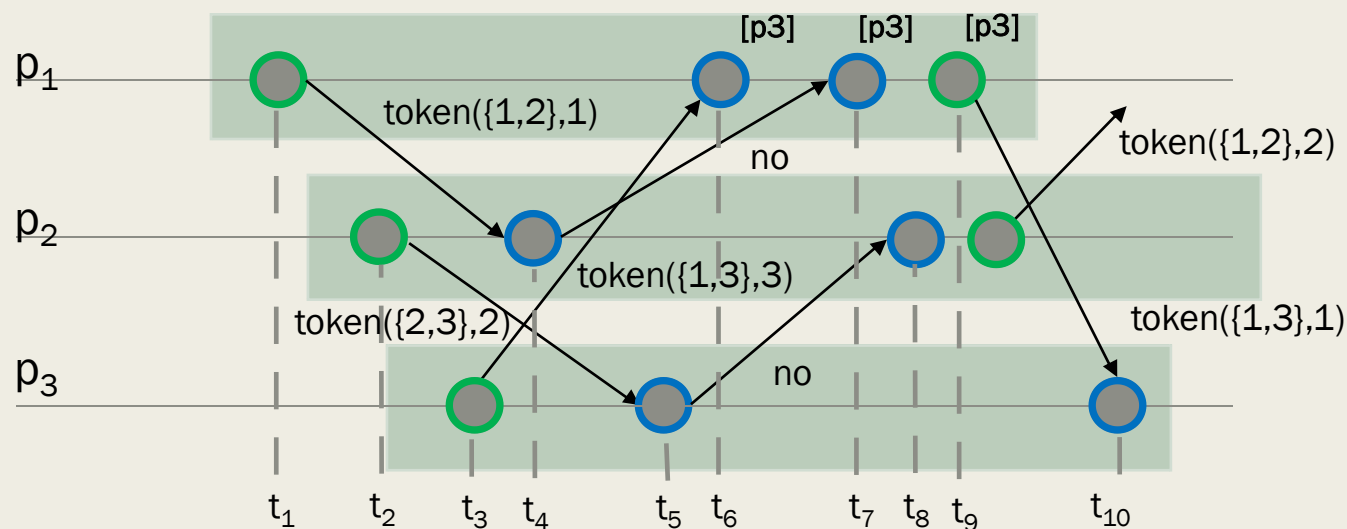
t_3 : proces p_1 nemá token pro komunikaci s žádným jiným partnerem, čeká

t_4 : proces p_2 vstupuje do kontextu, vybírá si p_1 , token má, a posílá jej partnerovi

t_5 : proces p_1 má zájem, nečeká na žádnou odpověď na svou žádost, přijímá

t_6 : proces p_2 ví, že komunikace je potvrzena ... **rendezvous**, pak opouští oba kontext

Synchronizace rendezvous procesů, příklad



t_1, t_2, t_3 : procesy vstupují do kontextu, vybírají si partnera, pro kterého drží token a ten odesílají
 t_4, t_5 : procesy s nižší prioritou p_1 a p_2 odpovídají 'no', a token si ponechávají
 t_6 : proces s vyšší prioritou p_1 neodpovídá, ale značí si alternativní možnost interakce
 t_9 : proces p_1 obdrží 'no' na svoji původní návrh a bere zvděk alternativní nabídce, posílá token
 t_{10} : odesílatel alternativního návrhu obdrží token, **rendezvous**

NEDOCHÁZÍ K DEADLOCKU, ANI K ODMÍTNUTÍ INTERAKCE VŠEMI PROCESY

ADA - programovací jazyk

- Vyvíjen od 80. let jako 'komplexní' jazyk poskytující, robustní a obecný nástroj pro programování systémů
- Strukturovaný, objektový, výjimky, AGOL + PASCAL + ...
- Zahrnující vše, co se zdá být užitečné, ale přesto není stříbrnou kulkou, která vyřeší vše, co je třeba (nejen vlkodlaka)

(viz např zde <http://www.cs.unc.edu/techreports/86-020.pdf>)



ADA LOVELACE

ADA

- ADA pro synchronizaci používá systém 'rendezvous' (aktivní zprávy)
 - *V Ada je úloha jedním sekvenčním procesem, který má lokální data.*
 - *Úlohy komunikují vyvoláním vstupní procedury jiné úlohy.*
- A má rendezvous s těmito úlohami.
 - *Klientská úloha vyvolává vstupní bod který jej může 'akceptovat'*
- Úloha na straně serveru a způsobí, že mají oba procesy rendezvous.
 - *Více akceptovatelných událostí může být na straně serveru očekáváno, pokud je použit příkaz "select";*

ADA - Accept

- Příkaz accept má tvar

```
"accept" <entry-name>  
[<formal parameter list>  
["do" <statements> "end"];
```

- Příkaz je proveden jen pokud jej vyvolá vzdálený proces

entry-name (s parametry)

- *Po provedení příkazu "end", výsledky jsou předány a oba procesy pokračují paralelně.*
- *Oba procesy, jak volající, tak přijímající, mohou být blokovány, dokud druhý proces není připraven*

ADA – Select, strážci

- Příkaz Select má následující tvar

– "select"

```
[ "when" <boolean-expresssion=> ]
    <accept-statement>
[ <statements> ]
{ < "or" [ "when" <boolean-expression=> ]
    <accept-statement }
[ <statements> ]
"else" [ <statements> ]
"end select;"
```

- 1. Vyhodnotí všechny *boolovské* výrazy, označí všechny “accept” příkazy se splněnou podmínkou jako otevřené *open*. Pokud podmínka není uvedena, pak je *accept* vždy otevřený.
- 2. Zvolte nějaký otevřený "accept" z volaného místa, pokud je otevřených acceptů více, zvolte nějaký nedeterministicky. Pokud žádný accept otevřený není, vykonejte ‘else’ část.
- 3. Pokud tu není žádný "else" a také žádný otevřený "accept,,", pak je vyvolána výjimka.

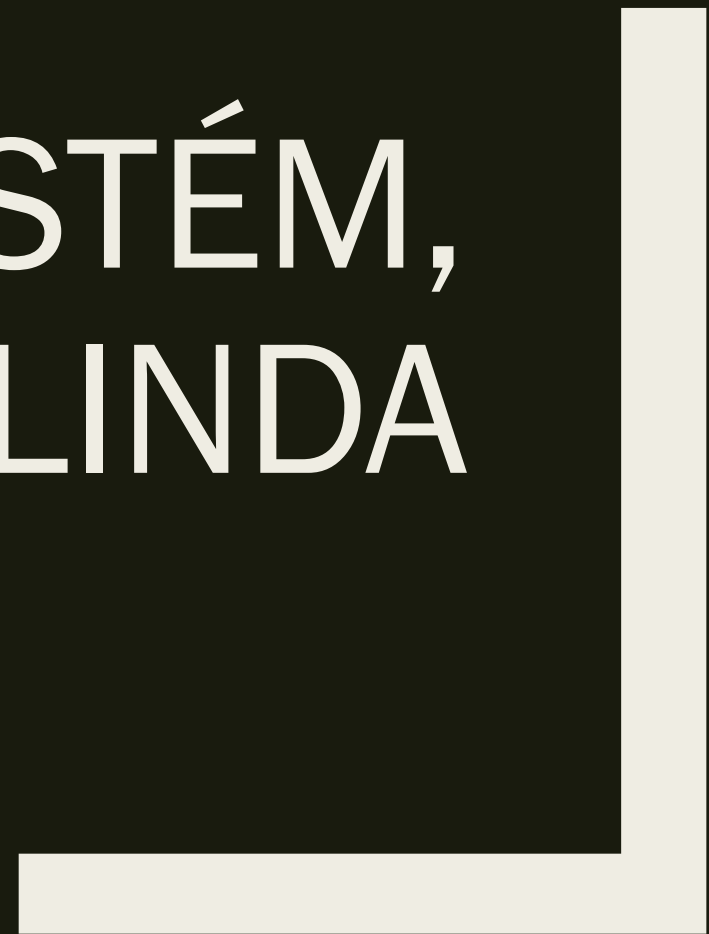
ADA, příklad, omezený buffer

```
task body bounded_buffer is
    buffer: array[0..9] of item;
    in,out: integer;
    count: integer;
    in := 0;
    out := 0;
    count := 0;
    [JÁDRO]
end bounded_buffer;
```

ADA, příklad, omezený buffer [JÁDRO]

```
begin loop
  select
    when count < 10 =>
      accept insert( it: item )
        do buffer[in mod 10] := it;
          in := in + 1;
          count := count - 1;
        end;
    or when count > 0 =>
      accept remove( it: out item )
        do it := buffer[out mod 10];
          out := out + 1;
          count := count - 1;
        end;
  end select;
end loop;
```


NÁSTĚNKOVÝ SYSTÉM, LINDA



LINDA - úvod



- Vyvinuta AT&T Bell Labs, Yale University (Ahuja, Carriero, Gelenter, LINDA & FRIENDS, 1986)
- Host programming language + LINDA -> Jazyk pro programování paralelních systémů
- Platformě a aplikačně nezávislé
- Původně vyvinuta C-Linda, Fortran-Linda
- Nyní například v Sicstus PROLOG, Agilla (agenti pro WSN)

LINDA

- Paralelní programování založené na jazyce C
- Jedná se o *koordinační jazyk* pro *asynchronní* a *asociativní* komunikační mechanismus nasíleným *globálním prostorem* nazývaným prostor n-tic *Tuples Space* (TS)
- Co je prostor n-tic?
 - *Virtuální sdílená paměť*
- Co je n-tice?
 - *Základní datová struktura nástěnky*
 - *Sekvence aktuálních a formálních položek*
 - *Příklad:*

`('arraydata', dim1, 13, 2)`

Nástěnka / prostor n-tic (TUPLES SPACE)



Generování n-tic

■ out

- *Generuje data (pasivní) n-tice*
- *Každá položka je vyhodnocena a umístěna na nástěnku*
- *Řízení je potom navrženo volajícím procesu*
- *Příklad: out ('array data', dim1, dim2);*

■ eval

- *Generuje procesy – aktivní n-tice*
- *Řízení je navrženo volajícím procesu ihned po aktivaci nového procesu n-ticí*
- *Každá položka je teoreticky vyhodnocena souběžně s ostatními a výsledek vložen do n-tice umístěné na nástěnce*
- *Položky obsahující funkci (nebo podproceduru) způsobí vznik nového procesu, který ji vykoná*
- *Příklad: eval ("test", f(i));*

Čtení a odstraňování n-tic

■ in

- *Používá šablony pro získání n-tic z nástěnky.*
- *Pokud je n-tice získána, je odstraněna z nástěnky a dále pak již není zde pro další přístupy.*
- *Pokud neexistuje n-tice, která by šabloně vyhovovala, je proces pozastaven. Takto lze dosáhnout synchronizace mezi procesy.*
- *Příklad: `in("arraydata", ?dim1, ?dim2);`*

■ rd

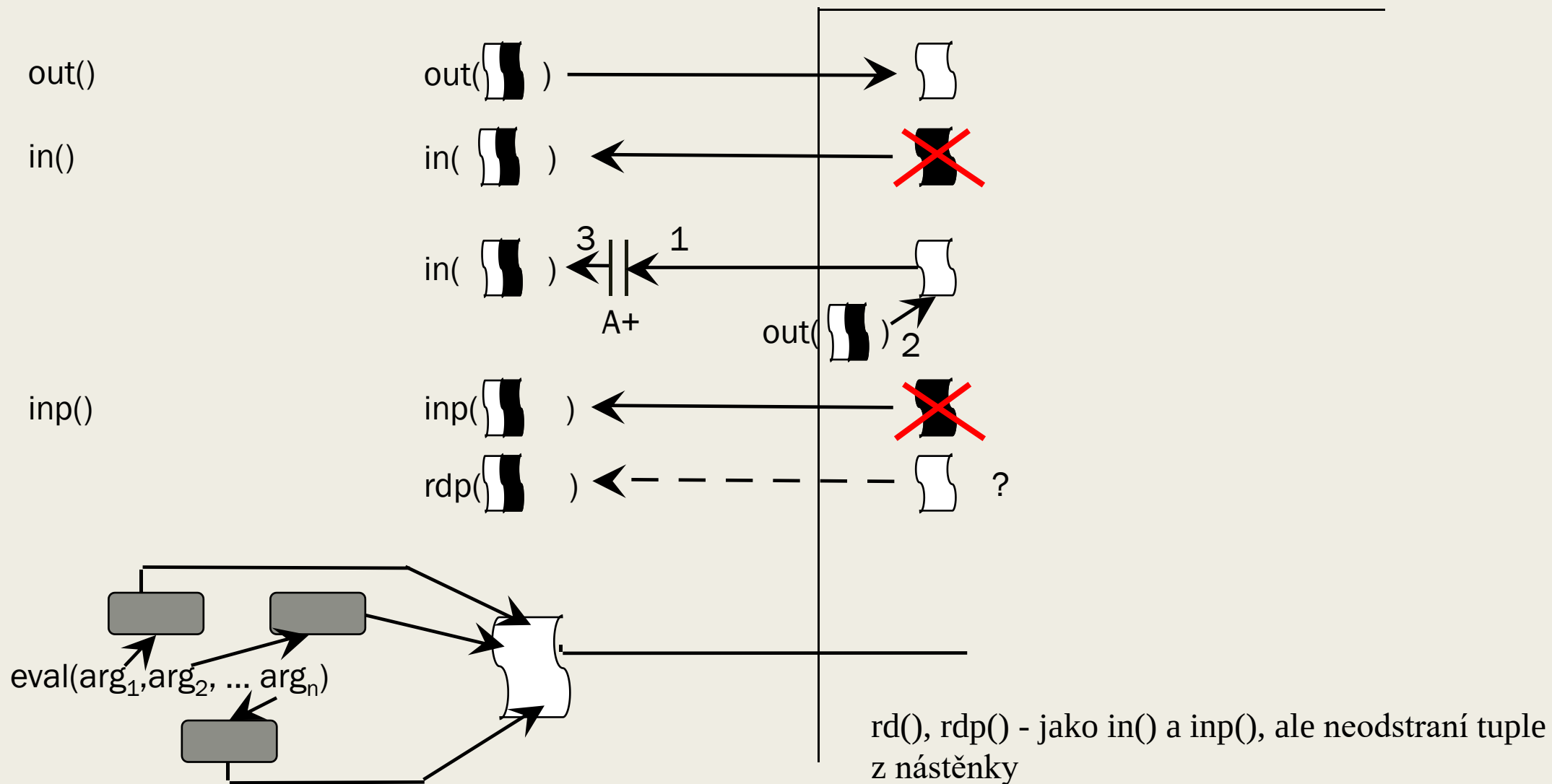
- *Používá šablonu pro získání dat bez jejich odstraňování z nástěnky.*
- *Po přečtení je n-tice nadále dostupná ostatním procesům.*
- *Pokud neexistuje n-tice, která by šabloně vyhovovala, je proces pozastaven.*
- *Příklad: `rd("arraydata", ?dim1, ?dim2);`*

Porovnání n-tice se šablonou

- Musí odpovídat počet položek v šabloně a hledané n-tici
 - *Také musí mít odpovídající typ, délku, hodnoty*
- Na odpovídající položce musí
 - *Musí odpovídat typ a délka prvků v šabloně s vyhledanou n-ticí*
 - *Pozor – pořadí vyhodnocování položek není definováno, proto něčemu jako ...*

```
out ("string", i++, i); // bychom se měli vyhnout
```
 - *Pokud je možné šabloně přiřadit více n-tic, není definováno, jak vybrat jednu konkrétní.*

Operátory jazyka LINDA



Standardní synchronizační primitiva v LINDA

■ *Semafor*

- P (Sem1) in(Sem1)
- V (Sem1) out(Sem1)

■ *Sdílená paměť*

- Reading
- Writing (atomic)

rd(Variable, ?value)
in(Variable, ?value)
out(Variable, NewValue)

■ *Asynchronní kanál*

- Send(Chan1, Val)
- Receive(Chan1, Val)
- Ready_to_Receive(Chan1)

out(Chan1, val)
in(Chan1, ?val)
rdp(Chan1, ?Dummy)

■ *Synchronní kanál*

- Send(Chan1, Val)
- Receive(Chan1, Val)

out(Chan1, value, Val)
in (Chan1, ?got)
in(Chan1, value, ?Val)
out(Chan1, got)

Příklad – pět filosofů

```
phil (i) {  
    while (1) {  
        think ();  
        in("chopstick", i);  
        in("chopstick", (i+1) % Num);  
        eat();  
        out ("chopstick", i);  
        out ("chopstick", (i+1) % Num);  
    }  
}
```

Lístkový algoritmus

```
int incr (name) {  
    in (name, ?n);  
    out (name, n++);  
    return n;  
}  
  
{  
    ticket = incr ("tick");  
    rd ("next", ?ticket);  
    ...  
    incr ("next");  
}
```

Vyhledání prvočísel

```
is_prime (me) {
    limit = sqrt (me) + 1;
    for (i=2; i<limit; i++) {
        rd ("primes", i, ?ok);
        if (ok && (me % i==0)) return 0;
    }
    return 1; }

main () {
    for (i=2; i<=LIMIT; i++) eval ("primes", i, is_prime(i));
    for (i=2; i<=LIMIT; i++) {
        rd ("primes", i, ?ok);
        if (ok) count++;
    }
    print ("%d. \n", count); }
```

Distribuované struktury

■ Seznam *(Name, Unique_identifier, Next, Val)*

- Průchod seznamem

```
typedef ... TID;
TID id, next;
rd(„list“, „head“, ?id)
while (id != ID_NIL) {
    rd(„list“, id, ?next, ?val);
    id = next;
}
```

■ Vytvoření seznamu

```
TID id, next;
next = ID_NIL;
while (val=getval()) {
    id=GetUniqueId();
    out(„list“, id, next, val);
    next = id;
}
out(„list“, „head“, next)
```

Distribuvované struktury

■ *Bag (batoh)*

- Nerozlišitelné prvku
- Příklad - master-workers (farm) konstrukce používá „bag of tasks“:
- Master Worker

```
out („task“, Descript) in („task“, ?New_Task)
```

■ *Sdílená proměnná*

- in („var1“, ?val)
- out („var1“, NewVal)

■ *Pole*

- (Jméno, Index/Indicie, Hodnoty)
- Průchod polem

```
for (i=0; i < MAX, i++)  
rd („Array“, i, ?val);
```

LINDA v SICSTUS PROLOGu



Klient - server přístup

```
| ?- use_module(library('linda/server')).
```



```
| ?- linda.  
Server address is msi:'61610'
```

```
| ?- use_module(library('linda/client')).
```



```
| ?- linda_client(msi:'61610').
```



```
| ?- linda_client(msi:'61610').
```



```
| ?- linda_client(msi:'61610').
```

LINDA v SICSTUS PROLOGu

- Blokující operace

```
rd(?Tuple), in (?Tuple)
```

```
rd(?TupleList, ?Tuple), in_noblock(?TupleList, ?Tuple)
```

- Neblokující operace

```
rd_noblock(?Tuple), in_noblock(?Tuple)
```

- Bag (viz PROLOG) rd_bag_noblock

Na nástěnce: (jmeno muz, franta), (jmeno, zena, pavla),

(jmeno, zena, hana), (jmeno, zena, tereza),

```
|?- rd([(jmeno,stroj,X), (jmeno, zena,Y)], JMENO).
```

Y = pavla

JMENO = (jmeno,zena,pavla) ?

```
|?- bagof_rd_noblock(jmena(X,Y), (jmeno,X,Y), BAG).
```

BAG = [jmena(muz,franta), jmena(zena,pavla), jmena(zena,hana), jmena(zena,tereza)] ?

LINDA v SICSTUS PROLOGu



```
writepni(L,L).  
writepni(A,L):-rd_noblock((prvocislo,A)),write((prvocislo,A)),nl,L2 is  
A+1, writepni(L2,L).  
writepni(A,L):-L2 is A+1, writepni(L2,L).  
writepn(L):-writepni(2,L).
```

```
deletepn:-rd_noblock((prvocislo,A)),in((prvocislo,A)),deletepn.  
deletepn.
```

```
testpni(A,B):- A < B*B, out((prvocislo,A)).  
testpni(A,B):- 0== (A mod B).  
testpni(A,B):- C is B+1, testpni(A,C).
```

```
pn(A,A).  
pn(A,B):-testpni(A,2),C is A+1, pn(C,B).
```

```
do(L):-rd((pnstart,A)),D is A+L, pn(A,D).
```

MODELY KOMUNIKUJÍCÍCH PROCESŮ

CSP, π – kalkul, OCCAM

CSP – Communicating Sequential Processes

- Communicating Sequential Processes (CSP) je formální jazyk vyvinutý pro modelování paralelních systémů komunikujících předáváním zpráv.
- C. A. R. HOARE 1978
- Výchozí pro další systémy, jako jsou například π -kalkul nebo OCCAM

OCCAM

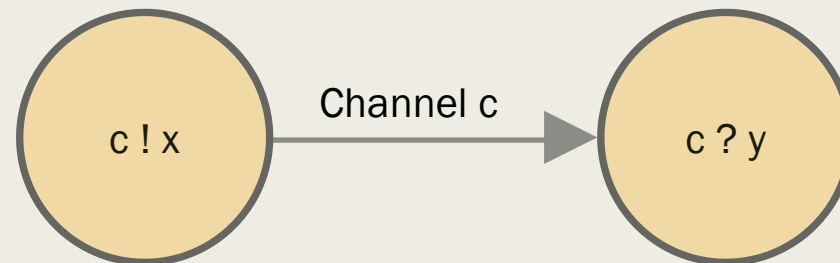
- Původně založen na Hoareho CSP, později inspirován a rozšířen o prvky z PI-kalkulu (OCCAM PI)
- Navržen tak, aby měl formální sémantikou vhodnou pro automatickou transformaci programu
- Transformace programů zapsaných v OCCAMu přímo na hardware (transputery, SW-HW codesign)
- Řada jazyků má k dispozici OCCAM – like rozšíření pro paralelní procesy, včetně JAVA

OCCAM, základní primitiva

- Occam primitivum je *proces* a je jednoho z následujících pěti druhů:

Přiřazení	<code>x := y + 2</code>
Vstup	<code>keyboard ? char</code>
Výstup	<code>screen ! char</code>
Skip	SKIP – NOP, které se ukončí
Stop	STOP -- NOP, které se nikdy neukončí

- Kanály poskytují možnost komunikace mezi procesy, nebufferovanou point-to-point synchronní komunikaci
- Kanály mají definovaný typ zpráv, které přenášejí



Sekvenční procesy

- SEQ executes sub-processes sequentially

SEQ

```
keyboard ? char -- read char from keyboard  
screen ! char -- write char to screen
```

Můžeme replikovat sekvence

```
SEQ i = 0 FOR array.size  
    stream ! data.array[i]
```

... je stejné jako

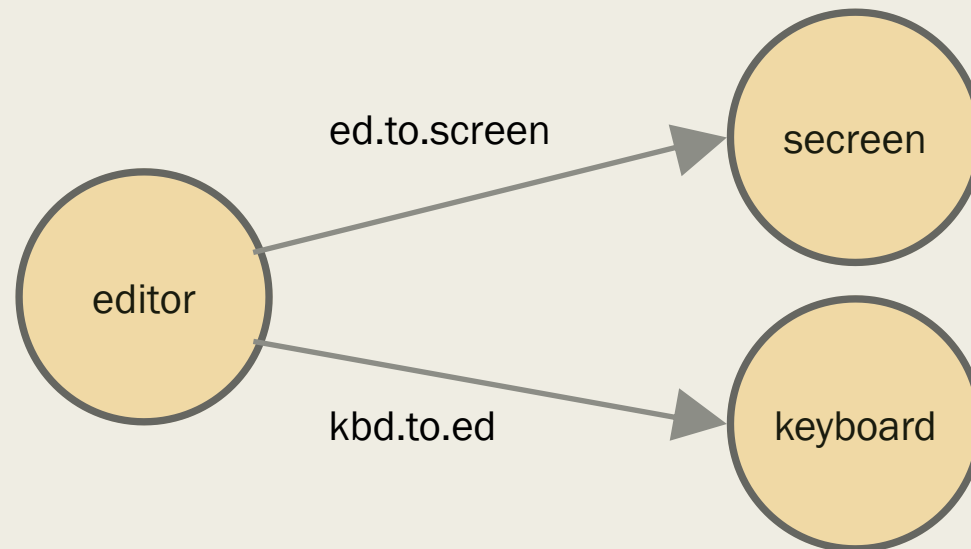
```
SEQ  
    stream ! data.array[0]  
    stream ! data.array[1]  
    ...
```

Kompozice paralelních procesů

- PAR spustí vykonávání paralelních procesů

PAR

```
keyboard(kbd.to.ed)  
editor(kbd.to.ed,ed.to.screen)  
screen(ed.to.screen)
```



PAR pro paralelní vykonávání procesů

■ Příklad: Square pipe

```
WHILE next <> EOF
  SEQ
    x := next
    PAR
      in ? next
      out ! x * x
```

■ Omezení přístupu paralelních procesů k datům

- *Variables modified in one arm of PAR cannot be read or written in other parts of PAR, e.g., PAR -- this PAR is invalid*

```
SEQ
  mice := 42 -- assigns to mice
  c ! 42
  c ? mice -- assigns to mice
```


Replikovaný PAR

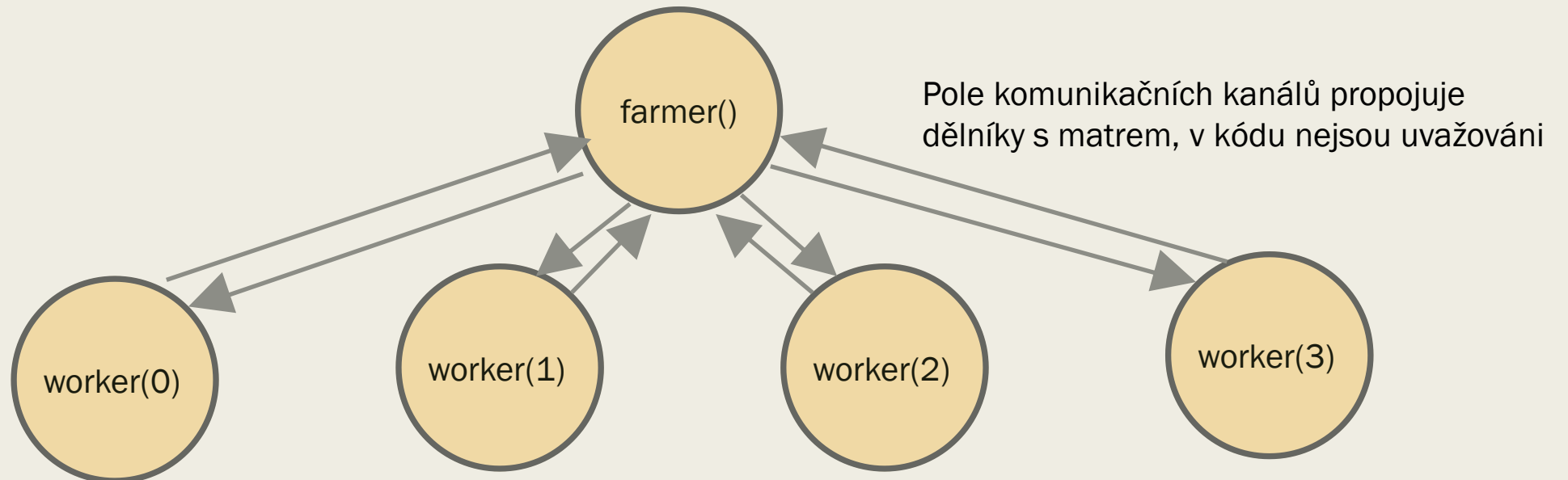
- Replikovaný PAR může být použit pro vytvoření pole paralelních procesů

PAR

```
farmer()
```

```
PAR i = 0 FOR 4 -- count must be constant
```

```
worker(i)
```

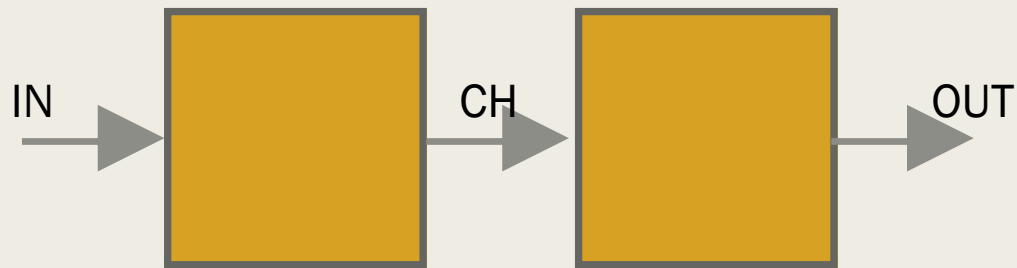


Vyrovnávací paměť (Buffer)



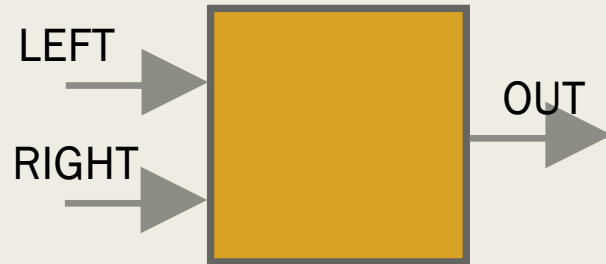
```
WHILE TRUE
  BYTE b:
  SEQ
    in ? b
    out ! b
```

Dvojíť Buffer



```
CHAN OF BYTE ch:
PAR
  WHILE TRUE
    BYTE b:
    SEQ
      in ? b
      ch ! b
  WHILE TRUE
    BYTE b:
    SEQ
      ch ? b
      out ! b
```

Dvojitý Buffer



```
WHILE TRUE
  left ? packet
  out ! Packet
  right ? packet
  out ! packet
```

```
WHILE TRUE
  PAR
    left ? packet
    out ! Packet
  right ? packet
  out ! packet
```

Alternace

- ALT combines a number of processes only one of which is executed
- Každý proces má strážce:
 - *vstup input on channel*
 - *čeká na časovač*
 - *Čeká na splnění boolovské podmínky*

ALT

```
left ? packet -- guard input statement
```

```
out ! packet
```

```
right ? packet -- guard input statement
```

```
out ! packet
```



Alternace

ALT

```
    strazcel  
        proces1
```

...

```
    strazcen  
        proces
```

Strážci mohou čekat na

1, vstup

```
    in ? data
```

2, platnost boolovské podmínky & vstup

```
    not.empty & in ? data
```

2, platnost boolovské podmínky & SKIP

```
    not.empty & SKIP
```

```
WHILE TRUE  
    ALT  
        left ? packet  
            out ! packet  
        right ? packet  
            out ! packet
```



OCCAM – deklarace procedur

-- other parts to program

```
PROC buff(CHAN OF BYTE in, out)
    WHILE TRUE
        BYTE x:
        SEQ
            in ? x
            out ! x :
```

-- end of buff

```
CHAN OF BYTE comms, buffer.in, buffer.out:
```

```
PAR
```

```
    buff(buffer.in, comms)
```

```
    buff(comms, buffer.out)
```

Π-KALKUL



π - kalkul

- Formální nástroj pro modelování paralelních a mobilních procesů
- Jazyk + axiomy + redukční pravidla
- Syntax:
 - Předpona π je v jednom z následujících tvarů $\pi = \bar{x}y \mid x(z) \mid \tau \mid [x=y]$
 - Proces je pak zapsán jako

$$P ::= M \mid P \mid P' \mid \nu z P \mid !P$$

$$M ::= \mathbf{0} \mid \pi.P \mid M + M'$$

π – kalkul, procesy

0	-	prázdný (ukončený) proces
$\pi.P$	-	předpona
$[x = y]P$	-	test
$P + P'$	-	nedeterministický výběr
$P \mid P'$	-	paralelní kompozice
$(\nu z)P$	-	omezení jména
$!P$	-	replikace

π – kalkul, redukční pravidla

Je možné přejmenovávat volná jména v procesu, nebo

(INTER)

$$\frac{}{\bar{x}z.P|x(y).Q \rightarrow P|Q[y/z]}$$

(aplikuje substituci na všechna volná jména v Q)

(R-PAR)

$$\frac{P \rightarrow Q}{P|R \rightarrow Q|R}$$

(R-RES)

$$\frac{P \rightarrow Q}{(\vartheta x)P \rightarrow (\vartheta x)Q}$$

(R-STRUCT)

$$\frac{P \equiv P' \rightarrow Q \equiv Q'}{P \rightarrow Q'}$$

(RESTRUCT)

$$\frac{}{\tau.P + M \rightarrow P}$$

(MATCH)

$$\frac{P \rightarrow Q}{[x=x]P \rightarrow Q}$$

π – kalkul, komunikace

- Synchronní přes pojmenovaný komunikační kanál

$$x(y).P | \bar{x}a.P'$$

Tj. Procesy komunikují přes komunikační kanál kanál x

$$x(y).P | \bar{x}a.P' \rightarrow P[y/a] | P'$$

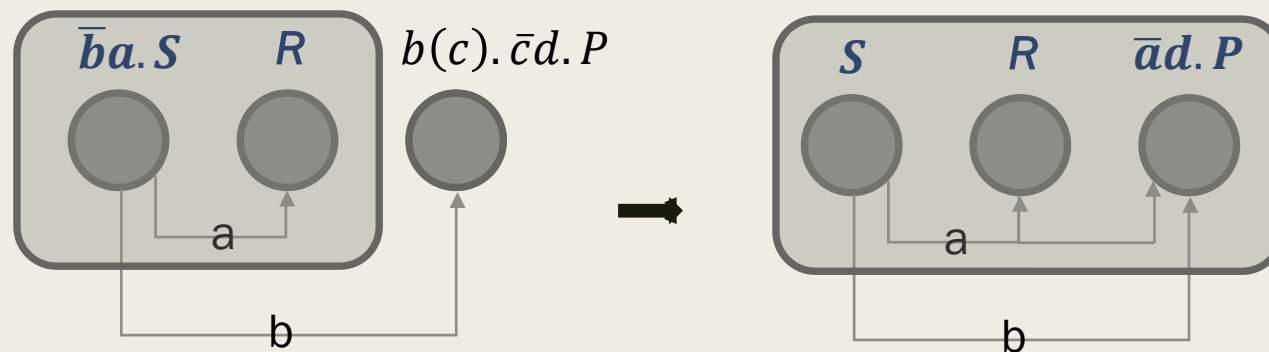
- Jiný příklad ukazuje mobilitu komunikujících kanálů:

$$\begin{aligned} & \bar{x}b. \mathbf{0} | x(y). \bar{y}a. \mathbf{0} | b(w). P | c(z). P' \rightarrow \\ & \mathbf{0} | \bar{b}a. \mathbf{0} | b(w). P | c(z). P' \equiv \\ & \bar{b}a. \mathbf{0} | b(w). P | c(z). P' \rightarrow \\ & \mathbf{0} | P[w/a] | c(z). p' \equiv P[w/a] | c(z). p' \end{aligned}$$

π – kalkul, omezené komunikační kanály

- Omezení se používá pro soukromé – skryté kanály nějaké skupiny procesů
- Na rozdíl od CSP, kde omezení nemohlo být rozšířeno, v π – kalkulaci mohou procesy v dosahu omezení poslat jméno skrytého kanálu vnějšímu procesu a rozšířit tak o něj toto omezení

$$(\exists a)(\bar{b}a.S|R)|b(c).\bar{c}d.P \rightarrow (\exists a)(S|R|\bar{a}d.P)$$



π – kalkul, sématica se štítkovanými přechody

- Provedení akce znázorňujeme štítkovanou přechodovou relací $\rightarrow^\alpha$ kde α je akce
- α může být:

xy	vstupní akce
$\bar{x}y$	otevřená výstupní akce
$\bar{x}(z)$	výstupní akce s omezeným jménem 'z'
τ	tichá akce

Příklad:

$$\bar{s}a. \bar{s}b. \mathbf{0} \rightarrow^{\bar{s}a} \bar{s}b. \mathbf{0}$$

$$a(y). Q \rightarrow^{az} Q[y/z]$$

- Tiché akce τ nejsou pozorovatelné (například i komunikace v rámci procesu, nedeterministický výběr apod.)

Příklad:

$$\bar{s}a. \bar{s}b. 0|z(y). 0|s(v). s(u). \bar{v}u. \mathbf{0} \rightarrow^\tau \bar{s}b. 0|z(y). 0|s(u). \bar{a}u. \mathbf{0}$$

π – kalkul, pozorování

- Pozorování se vztahují ke komunikačním kanálům, které nejsou soukromé pro nějakou skupinu procesů
- Zápis $P \downarrow x$ – process P má jméno x jako vstup
- Zápis $P \downarrow \bar{x}$ – process P má jméno x jako výstup
- Procesy jsou pozorovatelem neroznatelné, pokud mají stejné chování na výstupech
- Příklad:

$$P = (\exists z)(\bar{s}a.\bar{t}b.\mathbf{0}|z(y).\mathbf{0}|q(v).t(u).\bar{v}u.\mathbf{0}|\tau.w(z).\mathbf{0})$$

$P \downarrow a$??? – ne, je subjekt komunikace, ne komunikační kanál (vstup / výstup)

$P \downarrow \bar{s}$??? – ano, agent může poslat jméno po kanálu s

$P \downarrow \bar{t}$??? – ne, agent nyní nemůže posílat jméno po kanálu t

$P \downarrow q$??? – ano, agent může přijmout jméno po kanálu q

$P \downarrow t$??? – ne, agent nyní nemůže přijímat jméno po kanálu t

$P \downarrow z$??? – ne, jméno z je privátní procesu a není pozorovatelné z vnějšku

$P \downarrow w$??? – ne, agent nyní nemůže použít w, ale platí $P \Downarrow w$, viz dále

π – kalkul, kontext, kongruence

- Kontext zapisujeme s uvedením místa $[\cdot]$ na místo $\mathbf{0}$ procesu, který není degenerující (jediný na některé pozici nedeterministického výběru)
- Proces P , který má být proveden v **kontextu** C pak $C[P]$
- Procesy jsou v relaci kongruence, pokud jsou v této relaci pro libovolný kontext

$$\begin{aligned}C_0 &: (\vartheta z)([\cdot] \mid !z(w).\bar{w}a.\mathbf{0}) \\C_1 &: x(z).!(\vartheta w)(z(w).[\cdot] + y(v).\mathbf{0})\end{aligned}$$

- Aplikací procesu

$$\begin{aligned}C_0[!\bar{z}b.\mathbf{0}] &= (\vartheta z)(!\bar{z}b.\mathbf{0} \mid !z(w).\bar{w}a.\mathbf{0}) \\C_1[!\bar{z}b.\mathbf{0}] &= x(z).!(\vartheta w)(z(w).!\bar{z}b.\mathbf{0} + y(v).\mathbf{0})\end{aligned}$$

- **Kongruence** je relace S , kde $(P, Q) \in S \Rightarrow (C[P], C[Q]) \in S$ pro jakýkoliv kontext C

Strukturální kongruence

V relaci strukturální kongruence jsou procesy, které jsou stejné, akorát jinak zapsány.

V π -kalkulu je strukturální kongruence nejmenší kongruencí, která splňuje axiomy $\Rightarrow$

$$M + \mathbf{0} \equiv M$$

$$M_1 + M_2 \equiv M_2 + M_1$$

$$M_1 + (M_2 + M_3) \equiv (M_1 + M_2 + M_3)$$

$$P|\mathbf{0} \equiv P$$

$$P_1|P_2 \equiv P_2|P_1$$

$$(P_1|P_2)|P_3 \equiv P_1|(P_2|P_3)$$

$$!P \equiv P|!P$$

$$(\vartheta a)\mathbf{0} \equiv \mathbf{0}$$

$$(\vartheta a)(\vartheta b)P \equiv (\vartheta b)(\vartheta a)P$$

$$[x = x] \pi.P \equiv \pi.P$$

$$P_1|(\vartheta a)P_2 \text{ pokud } a \in fn(P_1)$$

π – kalkul, kongruence

- Kongruence je ekvivalence, která platí pro aplikaci jakékoliv substituce

Například procesy

$$P_1 = a(a)|\bar{b}b \quad a \quad P_2 = a(a).\bar{b}b + \bar{b}b.a(a)$$

nejsou kongruencí, protože pro substituci $\sigma = \{[b/a]\}$

$$P_1\sigma = b(b)|\bar{b}b \quad a \quad P_2\sigma = b(b).\bar{b}b + \bar{b}b.b(b)$$

Kongruencí jsou procesy

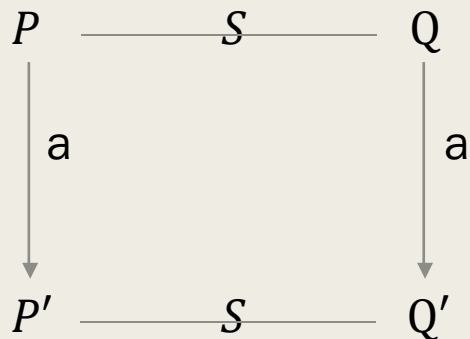
$$P_1 = a(a)|\bar{b}b \quad a \quad P_2 = a(a).\bar{b}b + \bar{b}b.a(a) + [a = b] \tau$$

Simulace

Relace simulace S , procesy P a Q

Silná simulace

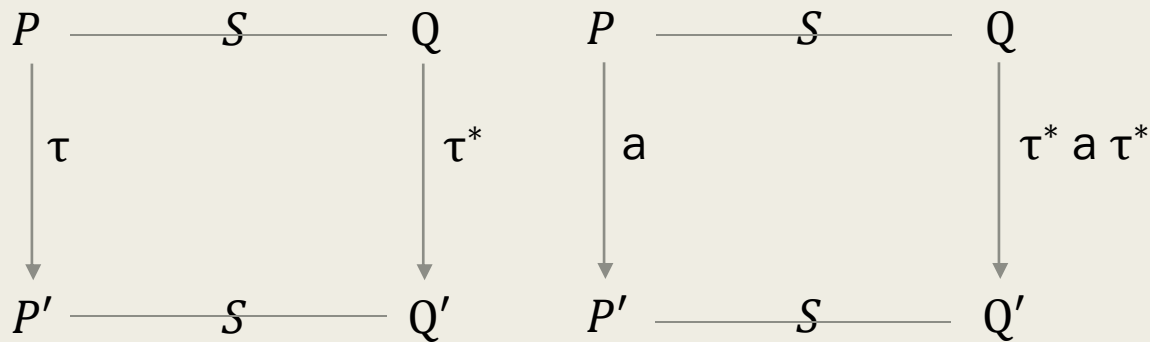
Pokud $P \mathrel{S} Q$ a $P \xrightarrow{a} P'$ pak existuje Q' takové, že $Q \xrightarrow{a} Q'$ a $P' \mathrel{S} Q'$



Slabá simulace

Pokud $P \mathrel{S} Q$ a $P \xrightarrow{\tau} P'$ pak existuje Q' takové, že $Q \xrightarrow{\tau^*} Q'$ a $P' \mathrel{S} Q'$

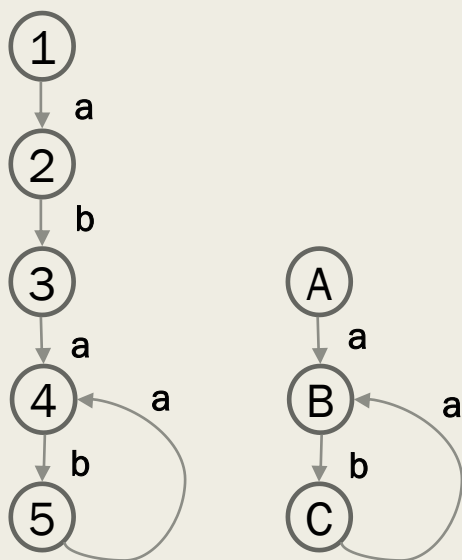
Pokud $P \mathrel{S} Q$ a $P \xrightarrow{a} P'$ pak existuje Q' takové, že $Q \xrightarrow{\tau^* a \tau^*} Q'$ a $P' \mathrel{S} Q'$



Bisimulace, bisimilarita

Bisimulace je relace taková, že procesy se mohou simulovat navzájem

Bisimilarita je relace mezi dvěma procesy, pro které existuje (alespoň jedna) relace simulace, je sjednocením všech bisimulací



Bisimulace

$S1 = \{(1,A), (2,B), (3,C), (4,B), (5,C), (A,1), (B,2), (C,3), (B,4), (C,5)\}$

$S2 = \{(1,C), (2,B), (3,C), (4,B), (5,C), (C,1), (B,2), (C,3), (B,4), (C,5)\}$

$S3 = \{(2,B), (3,C), (4,B), (5,C), (B,2), (C,3), (B,4), (C,5)\}$

$S4 = \{(3,A), (4,B), (5,C), (A,3), (B,4), (C,5)\}$

$S5 = \{(3,C), (4,B), (5,C), (3,C), (B,4), (C,5)\}$

$S6 = \{(4,B), (5,A), (5,C), (B,4), (A,5), (C,5)\}$

Bisimilarita

$R = \{(1,A), (1,C), (2,B), (3,A), (3,C), (4,B), (5,A), (5,C),$
 $(A,1), (C,1), (B,2), (A,3), (C,3), (B,4), (A,5), (C,5)\}$

π – kalkul, 'barbed' simulace, bisimulace

Centrem zájmu v oblasti procesních algeber je podobnost procesů

Jeden proces může simulovat jiný proces (relace **simulace**), založena na pozorováních -> **barbed simulace**

Silná 'barbed' bisimulace ->

Procesy P a Q jsou v relaci silné bisimulace $S(P, Q)$ pokud

$P \downarrow x$ implikuje $Q \downarrow x$

$P \rightarrow^\tau P'$ implikuje $Q \rightarrow^\tau Q'$ pro nějaké Q' a $S(P', Q')$

$Q \downarrow x$ implikuje $P \downarrow x$

$Q \rightarrow^\tau Q'$ implikuje $P \rightarrow^\tau P'$ pro nějaké Q' a $S(P', Q')$

Slabá 'barbed' bisimulace -> tiché akce nejsou pozorovatelné, proto použijeme relaci pro pozorovatelné akce $\Rightarrow^\alpha$

Jednou nebo více tichými akcemi přejde proces do stavu dle relace.

$$\Rightarrow \stackrel{\text{def}}{=} (\rightarrow^\tau)^*$$

Pak pro akci α

$$\Rightarrow^\alpha \stackrel{\text{def}}{=} \Rightarrow \rightarrow^\alpha \Rightarrow$$

Zápis $P \Downarrow x$ značí, že proces P může tichými akcemi přejít $\Rightarrow$ do stavu, kde $P \downarrow x$

π – kalkul, silná simulace, příklad

Pozor! Mohou být procesy P a Q v relaci silné bisimulace?

$$P = \tau. (a(x)b(y) + a(x)c(z))$$

$$Q = \tau. a(x)b(y) + \tau. a(x)c(z)$$

Může Q simulovat krok?

$$P \rightarrow^\tau P'$$

$$\tau. (a(x)b(y) + c(x)c(z)) \rightarrow^\tau a(x)b(y) + a(x)c(z)$$

Nemůže, protože

$$Q \rightarrow^\tau Q'$$

$$\text{bud' } 1, \tau. a(x)b(y) + \tau. a(x)c(z) \rightarrow^\tau a(x)b(y)$$

$$\text{nebo } 2, \tau. a(x)b(y) + \tau. a(x)c(z) \rightarrow^\tau a(x)c(z)$$

Pak v případě 1 $P' \rightarrow^{a(x)} P'' = c(z)$ nemůže $Q' \rightarrow^{a(x)} Q'' = b(y)$,

$$\text{protože } \neg(P'' \downarrow c \rightarrow Q'' \downarrow c) \text{ a } \neg(Q'' \downarrow b \rightarrow P'' \downarrow b)$$

V případě 2 $P' \rightarrow^{a(x)} P'' = b(y)$ nemůže $Q' \rightarrow^{a(x)} Q'' = c(z)$

$$\text{protože } \neg(P'' \downarrow b \rightarrow Q'' \downarrow b) \text{ a } \neg(Q'' \downarrow c \rightarrow P'' \downarrow c)$$

π – kalkul, ‘late’ / ‘early’ simulace

- ‘pozdní’ **late** či ‘včasná’ **early** simulace, založená na štítkovaných přechodech
- Liší se v tom, jak jsou chápány vstupní akce a to se týká převážně procesů s nedeterministickým výběrem.
- Pro obě **late/early** simulace S:

Pokud $P \xrightarrow{\alpha} P'$, akce není vstupní $\alpha \neq x(y)$ a vázané jméno v α není volné v P nebo Q ($bn(\alpha) \notin fn(P, Q)$), pak pro nějaké Q' , $Q \xrightarrow{\alpha} Q'$ jsou tyto procesy v relaci **late** nebo **early** simulace $P' S Q'$

Vstupní akce pro **late** simulaci

Pokud $P \xrightarrow{x(y)} P'$, $y \notin fn(P, Q)$, pak pro nějaký proces Q' , $Q \xrightarrow{\alpha} Q'$ a pro jakékoliv jméno w jsou tyto procesy v relaci **late** simulace $P'[w/y] S Q'[w/y]$

Vstupní akce pro **early** simulaci

Pokud $P \xrightarrow{x(y)} P'$, $y \notin fn(P, Q)$, pak pro všechna jména w existuje nějaké Q' , $Q \xrightarrow{\alpha} Q'$ a tyto procesy jsou v relaci **early** simulace $P'[w/y] S Q'[w/y]$

π – kalkul, ‘late’/‘early’ simulace, příklad

Uvažujme dva procesy

$$P = x(y).\tau.O + x(y).O$$

$$Q = P + x(y).[x=y].\tau.O$$

Jsou tyto procesy v relaci late bisimulace?

Nejsou, při late bisimulaci nedokáže proces P simulovat krok $x(y)$ v případě, že proces Q vybere $x(y).[x=y].\tau.O$ podproces. Příjem jména x za vstupní y způsobí jiné chování (provedení τ) než příjem jiného jména za y (způsobí zablokování procesu, tedy O). Tedy P nedokáže jedním z možných podprocesů $x(y).\tau.O$ či $x(y).O$ nedokáže simulovat všechna možná přijímaná jména.

Jsou tyto procesy v relace early bisimulace?

Jsou, v tomto případě je potřeba, aby pro každé jméno přijímané přes $x(y)$ existoval podproces, který ‘odpovídá’ podprocesu druhého procesu. Tedy pokud by pro přijímané jméno proces Q (nedeterministicky) zvolil podproces s testem, tak P může reagovat na základě jména, jestli odpovídající jeho podproces je $\tau.O$ (v případě, že přijímané jméno je x a test tedy projde), nebo O

π – kalkul, příklady

- Forwarder $\text{FW}(a, b) = a(z). \bar{b}z$

$$\bar{a}x \mid \text{FW}(a, b) = \bar{a}x \mid a(z). \bar{b}z \rightarrow \bar{b}x$$

$$\begin{aligned} \bar{a}x \mid (\vartheta b)(\text{FW}(a, b) \mid \text{FW}(b, c)) &= \bar{a}x \mid (\vartheta b)(a(z). \bar{b}z \mid b(z). \bar{c}z) \rightarrow \\ &(\vartheta b)(\bar{b}x \mid b(z). \bar{c}z) \rightarrow \bar{c}x \end{aligned}$$

- Duplikátor $\text{D}(a, b, c) = a(d). (\bar{b}d \mid \bar{c}d)$

$$\bar{a}x \mid \text{D}(a, b, c) = \bar{a}x \mid a(d). (\bar{b}d \mid \bar{c}d) \rightarrow \bar{b}x \mid \bar{c}x$$

- Generátor nového jména $\text{NN}(a) = a(x). (\vartheta b)(\bar{x}b)$

$$\bar{i}o \mid \text{NN}(i) = \bar{i}o \mid i(o). (\vartheta b)(\bar{o}b) \rightarrow (\vartheta b)(\bar{o}b)$$

π – kalkul, příklad, neomezený buffer

Definujeme procesy modelující neomezený buffer

$$B(i, o) = i(x).C(x, i, o)$$

$$C(x, i, o) = \bar{o}x.B(i, o) + i(y).(C(y, i, o) \sim C(x, i, o))$$

$$C(y, i, o) \sim C(x, i, o) = (\vartheta m)(C(y, i, m) | C(x, m, o))$$

Prozkoumáme chování procesu

$$\bar{i}a.\bar{i}b.o(x).o(y).\bar{i}c.o(z) \mid B(i, o)$$

π – kalkul, příklad, neomezený buffer

$$\begin{aligned} B(i, o) &= i(x). C(x, i, o) \\ C(x, i, o) &= \bar{o}x.B(i, o) + i(y).(C(y, i, o) \sim C(x, i, o)) \\ C(y, i, o) \sim C(x, i, o) &= (\vartheta m)(C(y, i, m) | C(x, m, o)) \end{aligned}$$

$$\bar{i}a.\bar{i}b.o(x).o(y).\bar{i}c.o(z) \mid B(i, o) =$$

$$\begin{aligned} \bar{i}a.\bar{i}b.o(x).o(y).\bar{i}c.o(z) \mid i(x).C(x, i, o) &\rightarrow \\ \bar{i}b.o(x).o(y).\bar{i}c.o(z) \mid C(a, i, o) &= \end{aligned}$$

$$\bar{i}b.o(x).o(y).\bar{i}c.o(z) \mid \bar{o}a.B(i, o) + i(y).(C(y, i, o) \sim C(a, i, o)) \rightarrow$$

$$o(x).o(y).\bar{i}c.o(z) \mid (C(b, i, o) \sim C(a, i, o)) =$$

$$\begin{aligned} o(x).o(y).\bar{i}c.o(z) \mid (\vartheta m)(C(b, i, m) | C(a, m, o)) &= \\ o(x).o(y).\bar{i}c.o(z) \mid & \end{aligned}$$

$$(\vartheta m)(\bar{m}b.B(i, m) + i(y).(C(y, i, m) \sim C(b, i, m))) \mid \bar{o}a.B(m, o) + m(y).(C(y, m, o) \sim C(a, m, o))$$

π – kalkul, příklad, neomezený buffer

$$\begin{aligned} B(i, o) &= i(x). C(x, i, o) \\ C(x, i, o) &= \bar{o}x.B(i, o) + i(y). (C(y, i, o) \sim C(x, i, o)) \\ C(y, i, o) \sim C(x, i, o) &= (\vartheta m)(C(y, i, m) | C(x, m, o)) \end{aligned}$$

$$o(x). o(y). \bar{i}c. o(z) |$$

$$\begin{aligned} &(\vartheta m)(\bar{m}b.B(i, m) + i(y). (C(y, i, m) \sim C(b, i, m))) | \bar{o}a.B(m, o) + \\ &m(y). (C(y, m, o) \sim C(a, m, o)) \rightarrow \end{aligned}$$

$$o(y). \bar{i}c. o(z) | (\vartheta m)(\bar{m}b.B(i, m) + i(y). (C(y, i, m) \sim C(b, i, m))) | B(m, o) =$$

$$\begin{aligned} &o(y). \bar{i}c. o(z) | (\vartheta m)(\bar{m}b.B(i, m) + i(y). (C(y, i, m) \sim C(b, i, m))) | m(x). C(x, m, o) \rightarrow \\ &o(y). \bar{i}c. o(z) | (\vartheta m)(B(i, m) | C(b, m, o)) = \end{aligned}$$

$$o(y). \bar{i}c. o(z) | (\vartheta m)(i(x). C(x, i, o) | \bar{o}b.B(m, o) + i(y). (C(y, m, o) \sim C(b, m, o))) \rightarrow$$

$$\bar{i}c. o(z) | (\vartheta m)(i(x). C(x, i, o) | B(m, o))$$

π – kalkul, příklad, neomezený buffer

$$\begin{aligned}B(i, o) &= i(x). C(x, i, o) \\C(x, i, o) &= \bar{o}x.B(i, o) + i(y).(C(y, i, o) \sim C(x, i, o)) \\C(y, i, o) \sim C(x, i, o) &= (\vartheta m)(C(y, i, m) | C(x, m, o))\end{aligned}$$

$$\begin{aligned}\bar{1}c.o(z) | (\vartheta m)(i(x). C(x, i, o) | B(m, o)) &\rightarrow \\o(z) | (\vartheta m)(C(c, i, o) | B(m, o)) &= \end{aligned}$$

$$o(z) | (\vartheta m)(\bar{o}c.B(i, o) + i(y).(C(y, i, o) \sim C(c, i, o)) | B(m, o)) \rightarrow \mathbf{0} | (\vartheta m)(B(i, o) | B(m, o))$$